



ARQUITECTURA Y SISTEMAS OPERATIVOS

CLASE 5

PLANIFICACIÓN DE LA CPU



Planificación de la CPU

Hasta ahora vimos que un SO debe administrar múltiples recursos y coordinar la ejecución de varios procesos o hilos. Pero qué pasa cuando hay varios procesos listos, ¿quién usa la CPU?

La CPU es un recurso compartido y limitado, y aunque tengamos sistemas con varios núcleos de ejecución, sigue existiendo una decisión fundamental: **qué entidad se ejecuta, cuándo se ejecuta y durante cuánto tiempo**. Ese problema se conoce como **planificación de la CPU**.

La planificación es una de las funciones centrales de un SO.

El SO debe decidir:

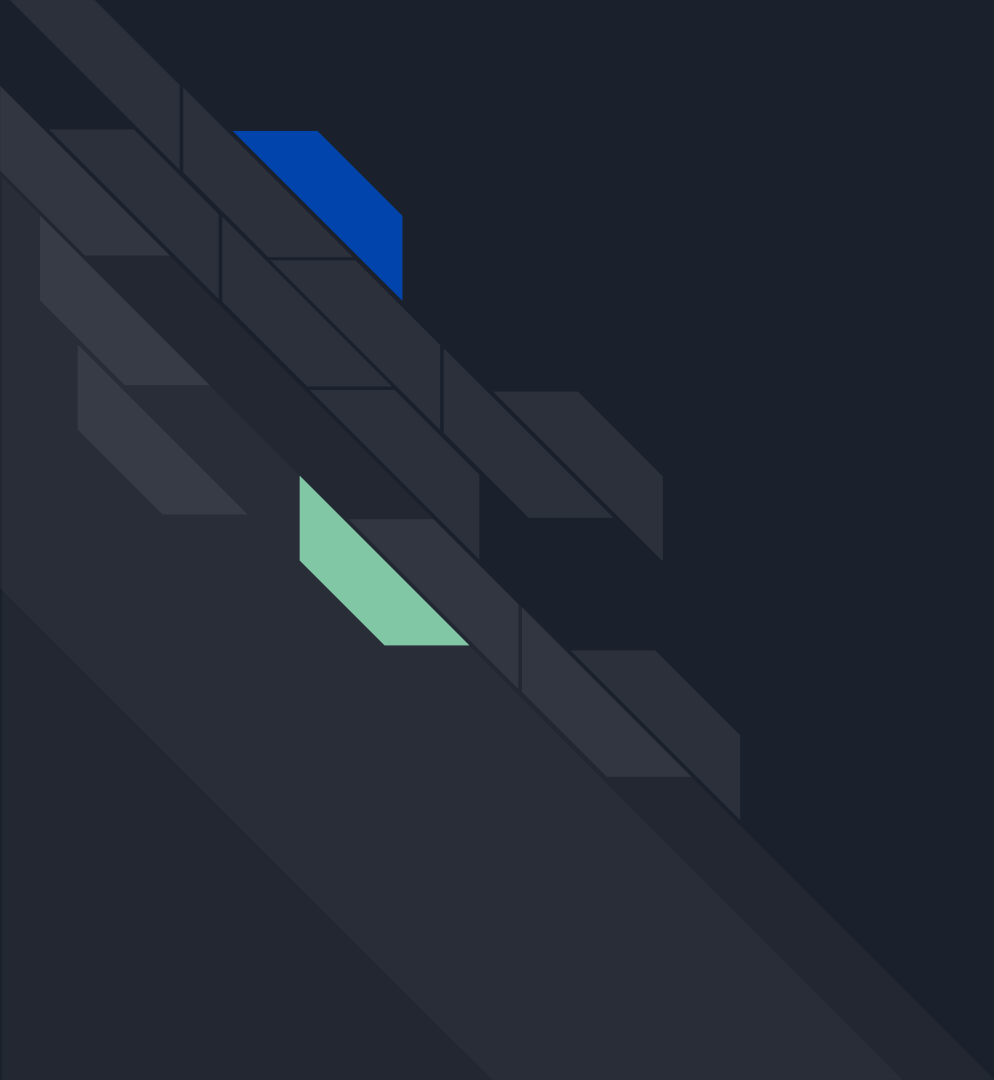
- Qué proceso ejecuta
- Cuándo se ejecuta
- Y durante cuánto tiempo



Planificación de la CPU

Estas decisiones afectan:

- Rendimiento
- Tiempos de espera
- Capacidad de respuesta
- Utilización del hardware.



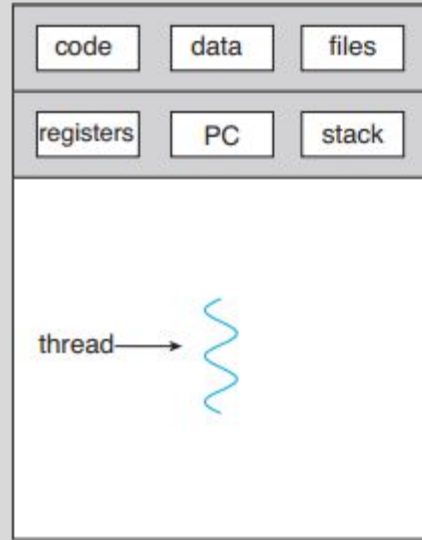
REPASO

Repaso: procesos, hilos y conurrencia

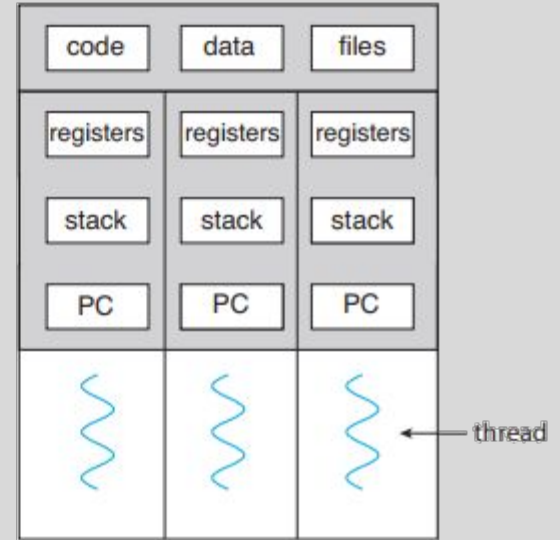
Un proceso puede contener uno o varios hilos.

Como los hilos comparten el mismo espacio de direcciones y otros recursos del proceso, permiten que distintas actividades de una misma aplicación avancen de manera concurrente.

En un sistema, puede haber varios procesos activos al mismo tiempo, y también varios hilos listos para ejecutarse.



single-threaded process



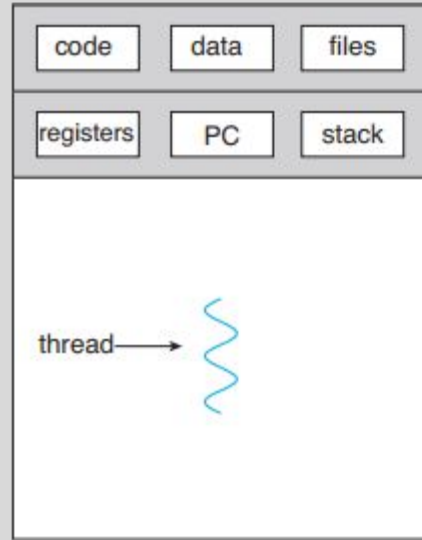
multithreaded process

Repaso: procesos, hilos y conurrencia

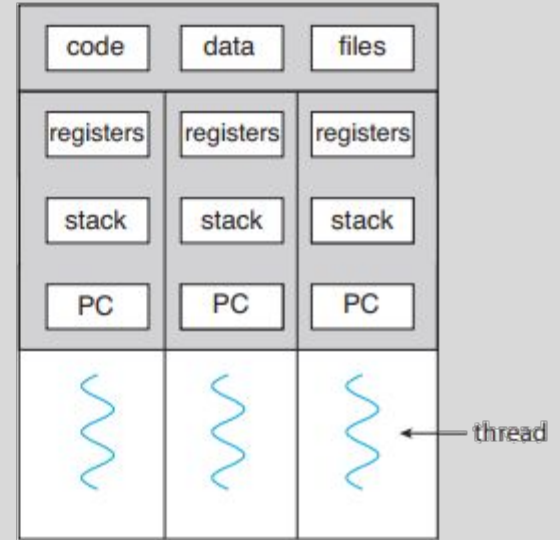
Desde el punto de vista del SO, puede haber múltiples candidatos listos para usar el procesador, pero no todos pueden ejecutarse al mismo tiempo.

Por eso, aunque la clase pasada estuvo centrada en qué son los hilos y por qué se usan, esta clase se enfoca en una pregunta diferente:

- Cuando hay varias unidades de ejecución listas, cómo decide el SO cuál ejecuta primero.

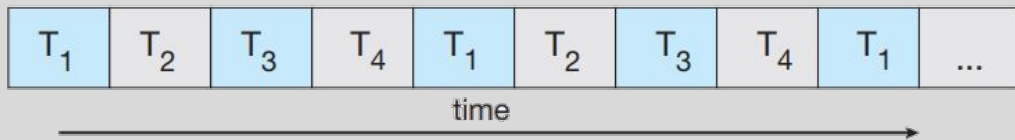


single-threaded process



multithreaded process

single core



Concurrent execution on a single-core system.



Parallel execution on a multicore system.

Del repaso al problema de planificación

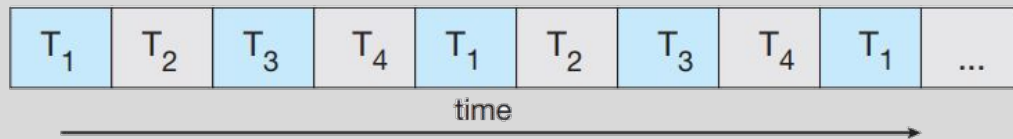
Tener múltiples procesos en memoria no garantiza que todos puedan avanzar al mismo ritmo.

La CPU es un recurso finito. En cada núcleo, en cada instante, solo puede ejecutarse una secuencia de instrucciones a la vez.

Si varios procesos están listos para ejecutar, el SO necesita aplicar algún criterio para decidir cuál.

Esa decisión no es arbitraria.

single core



Concurrent execution on a single-core system.



Parallel execution on a multicore system.

Del repaso al problema de planificación

Dependiendo del algoritmo elegido, puede cambiar mucho la experiencia del sistema:

- Algunos procesos pueden terminar antes.
- Otros pueden esperar demasiado.
- Algunas aplicaciones pueden responder mejor al usuario.
- Y otras pueden quedar perjudicadas si el criterio no las favorece.

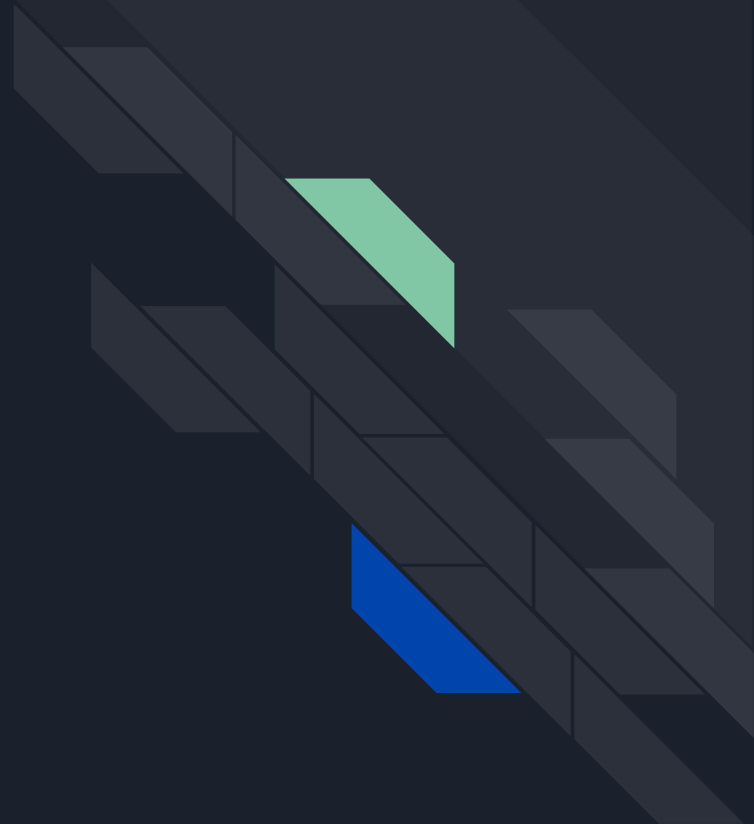
Planificar no es solamente “elegir el siguiente”: es definir una **política de reparto del tiempo de CPU**.



BIBLIOGRAFIA

- Operating System Concepts. By Abraham, Silberschatz.
 - Capítulo V

CONCEPTOS BÁSICOS





La CPU como recurso compartido

En un sistema multiprogramado, varios procesos conviven en memoria y todos compiten, en mayor o menor medida, por obtener tiempo de CPU.

El procesador es un recurso compartido, y obliga al SO a administrarlo con cuidado.

Objetivo general:

- Evitar que la CPU quede ociosa cuando hay trabajo útil por hacer.
- Repartir el tiempo de CPU con criterio.



Multiprogramación y aprovechamiento de la CPU

En sistemas simples, un proceso podía ejecutarse hasta necesitar una operación de E/S, y durante esa espera la CPU quedaba inactiva. Ese tiempo ocioso era tiempo perdido: el procesador no hacía trabajo útil mientras esperaba que otro dispositivo completara una operación.

La **multiprogramación** aparece para mejorar esa situación. Al mantener varios procesos en memoria al mismo tiempo, cuando uno debe esperar una operación de E/S, el SO puede retirarle la CPU y asignársela a otro proceso que esté listo para ejecutar.

La idea es:

- Mientras uno espera, otro avanza.

Esto mejora el aprovechamiento del procesador. Por eso, la **planificación de CPU** está muy ligada a la **multiprogramación**: sin una política de planificación, no habría forma ordenada de decidir qué proceso toma el control de la CPU cuando otro deja de ejecutar.

Modelo de ráfagas CPU-E/S

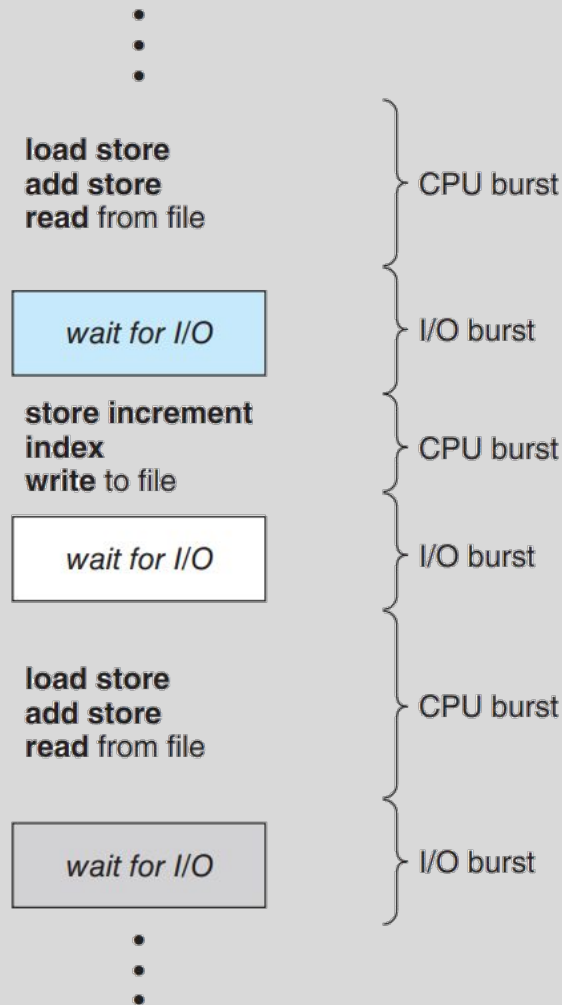
La planificación de CPU se apoya en una observación muy importante sobre cómo ejecutan los procesos.

En general, lo habitual es que un proceso alterne entre dos tipos de etapas:

- Períodos donde necesita procesador para ejecutar instrucciones.
- Períodos donde debe esperar una operación de E/S.

Es decir:

- Ráfagas de CPU
- Ráfagas de E/S

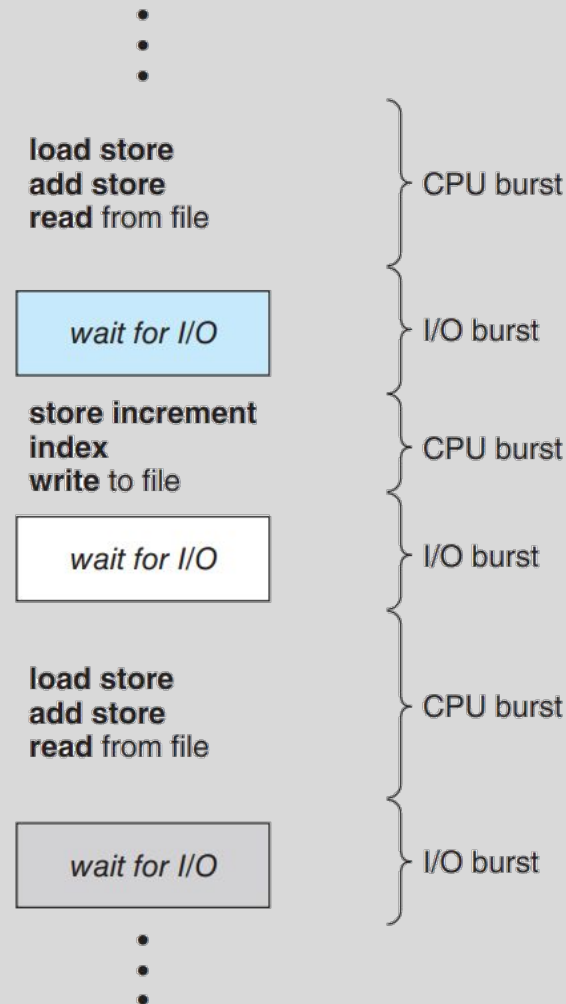


Modelo de ráfagas CPU-E/S

Así, la vida de un proceso suele verse como una secuencia alternada de:

- CPU
- Espera de E/S
- CPU
- Espera de E/S
- y así sucesivamente hasta finalizar.

Es importante entender que un proceso no usa CPU todo el tiempo, alterna entre trabajo y espera.





Cola de listos y planificador de CPU

Cada vez que la CPU queda disponible, el SO necesita seleccionar quién la usará a continuación.

Para eso existe la **cola de listos** (*ready queue*). Ahí se encuentran los procesos que están preparados para ejecutar, pero que todavía no tienen asignada la CPU.

El componente encargado de elegir uno de ellos es el **planificador de CPU** (*CPU scheduler*). Cuya función es seleccionar, entre todas las entidades listas, cuál será la próxima en ejecutar.

Desde el punto de vista conceptual, todos los elementos de la cola de listos están esperando la misma cosa: **tiempo de CPU**.



Cuándo decide el planificador

El SO no toma decisiones de planificación en cualquier momento.

Existen situaciones bien definidas en las que debe reconsiderar quién debe usar la CPU.

Básicamente, esas decisiones aparecen cuando ocurre alguno de estos eventos:

1. Un proceso pasa de ejecución a espera (ej. cuando solicita E/S).
2. Un proceso pasa de ejecución a listo (ej. cuando es interrumpido).
3. Un proceso pasa de espera a listo (ej. cuando termina su E/S).
4. Un proceso finaliza.



Cuándo decide el planificador

No todos estos casos implican el mismo grado de libertad para el SO. Por ejemplo:

- Si el proceso se bloquea o termina, necesariamente la CPU debe asignarse a otro.
- En cambio, si aparece una interrupción o vuelve a estar listo otro proceso, puede existir una decisión real entre continuar con el actual o reemplazarlo.

En base a esto, surge una diferencia central entre dos grandes esquemas:

- **Planificación apropiativa**
- **Planificación no apropiativa**



Planificación no apropiativa

Una vez que un proceso obtiene la CPU, la conserva hasta que ocurre una de estas dos cosas:

- Termina su ejecución
- O se bloquea voluntariamente (ej. al pedir una operación de E/S).

El SO **no le quita la CPU** mientras el proceso sigue ejecutando normalmente.

Ventaja: es simple de razonar y de implementar, reduce la cantidad de interrupciones y decisiones forzadas durante la ejecución.

Pero también tiene una **limitación** importante: si un proceso tarda mucho en liberar la CPU, los demás deben esperar, aunque alguno de ellos sea corto o interactivo.



Planificación apropiativa

En este esquema, el SO puede quitarle la CPU a un proceso aunque éste no haya terminado ni se haya bloqueado voluntariamente.

Eso significa que el sistema puede interrumpirlo y decidir que otro proceso use el procesador. Esto permite una administración más flexible del tiempo de CPU.

Por ejemplo:

- Puede favorecer tareas cortas
- Puede mejorar la capacidad de respuesta de aplicaciones interactivas
- Y puede evitar que un proceso largo monopolice el procesador durante demasiado tiempo.



Planificación apropiativa

La contracara es que el sistema debe intervenir más activamente:

- Hay más decisiones de planificación
- Más cambios de contexto
- Y más cuidado en la coordinación de datos compartidos

La planificación apropiativa es la base de la mayoría de los SO modernos, porque permite construir sistemas más reactivos y más aptos para cargas mixtas.



Riesgos y costos de la apropiación

La planificación apropiativa mejora la flexibilidad del sistema, pero no es gratis.

Cuando el SO puede interrumpir la ejecución de un proceso en cualquier momento, aparecen nuevos problemas de **coordinación**.

Uno muy importante es que pueden surgir **condiciones de carrera**, si varios procesos o hilos acceden a datos compartidos y uno de ellos es interrumpido en un momento crítico.

- Por ejemplo, si una estructura de datos está siendo modificada y el sistema quita la CPU antes de que la actualización termine, otro flujo de ejecución podría observar un estado intermedio o inconsistente.



Riesgos y costos de la apropiación

Esto no afecta solo a programas de usuario. También importa dentro del propio kernel, donde existen estructuras internas que deben mantenerse coherentes incluso frente a interrupciones y cambios de contexto.

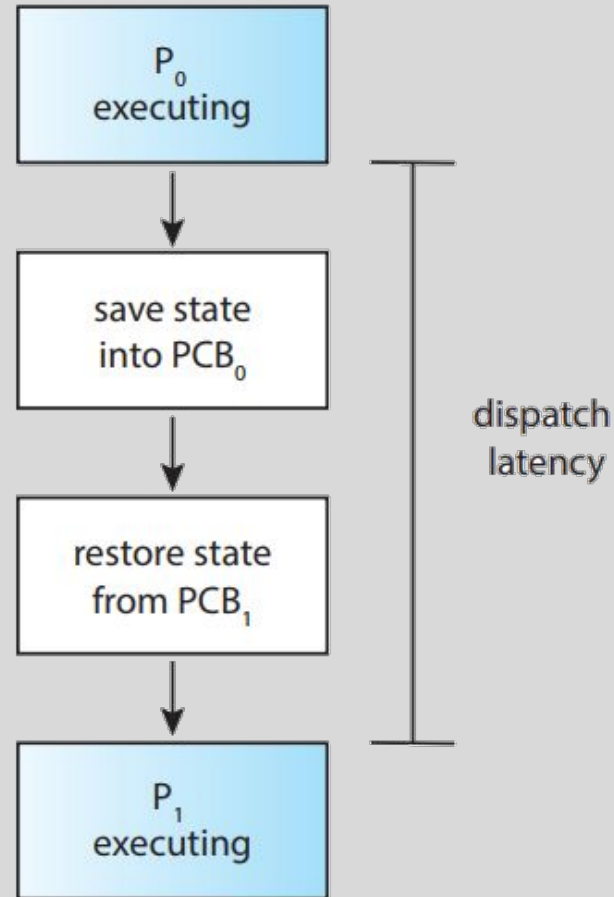
Por eso, un kernel apropiativo necesita mecanismos de protección, como:

- Exclusión mutua
- Regiones críticas
- O secciones donde ciertas interrupciones se deshabilitan temporalmente

Dispatcher y cambio de contexto

El planificador decide quién debería ejecutar. Pero una cosa es tomar la decisión, y otra ejecutarla.

El **dispatcher** es el componente del SO que hace esto: entrega el control de la CPU al proceso elegido por el planificador.

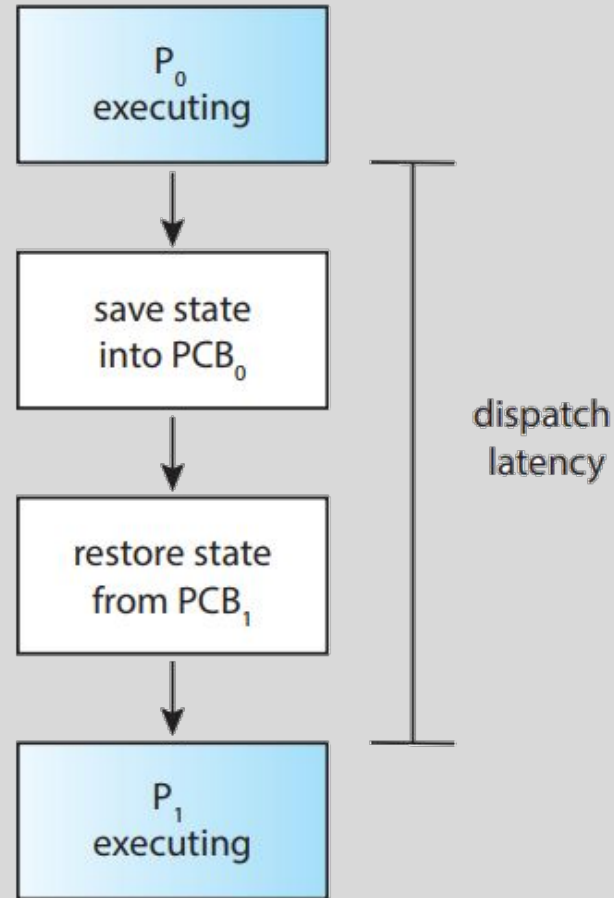


The role of the dispatcher

Dispatcher y cambio de contexto

Para hacerlo debe:

- Detener o suspender la ejecución actual
- Guardar el contexto necesario del proceso saliente
- Restaurar el contexto del proceso entrante
- Pasar al modo de ejecución correspondiente
- Y reanudar el programa en el punto donde debe continuar



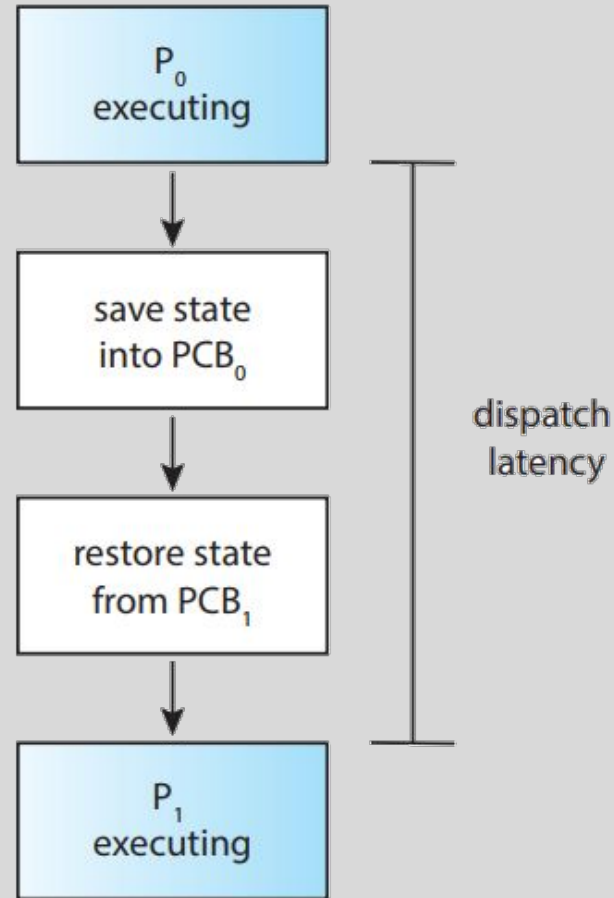
The role of the dispatcher

Dispatcher y cambio de contexto

Ese conjunto de operaciones forman parte del **cambio de contexto**.

El cambio de contexto es indispensable para alternar entre procesos, pero tiene un costo: durante ese tiempo, la CPU no está ejecutando trabajo útil del programa de usuario, sino trabajo administrativo del SO.

Por eso, aunque la planificación sea necesaria, no es gratuita.



The role of the dispatcher

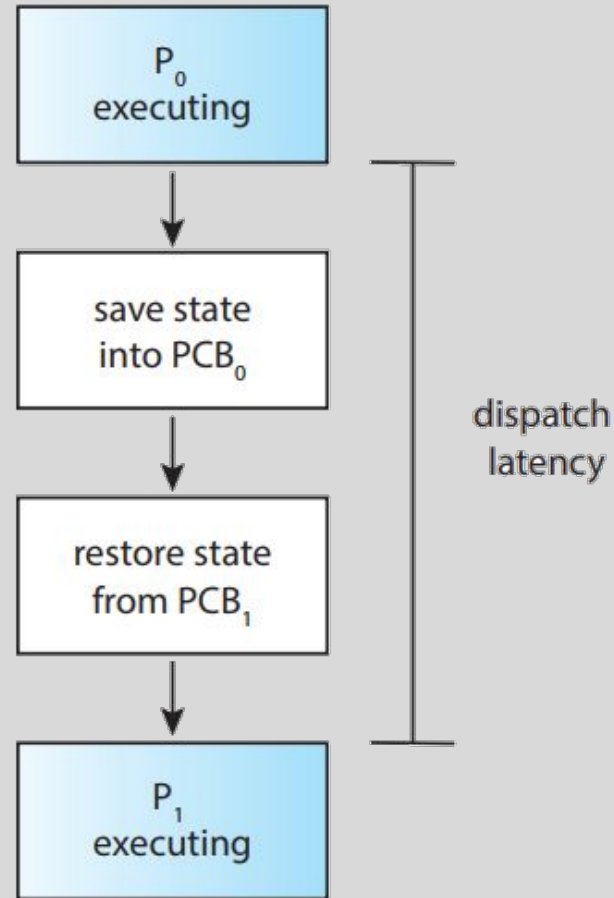
Latencia de despacho

Cada vez que el sistema cambia de una entidad de ejecución a otra, invoca al *dispatcher*.

Y el tiempo que tarda en completar esa operación, se lo conoce como **latencia de despacho**.

La latencia de despacho incluye:

- Detener el proceso actual
- Guardar su estado
- Cargar el estado del siguiente
- Preparar la CPU para reanudarlo
- Y devolver el control a la ejecución correspondiente



The role of the dispatcher

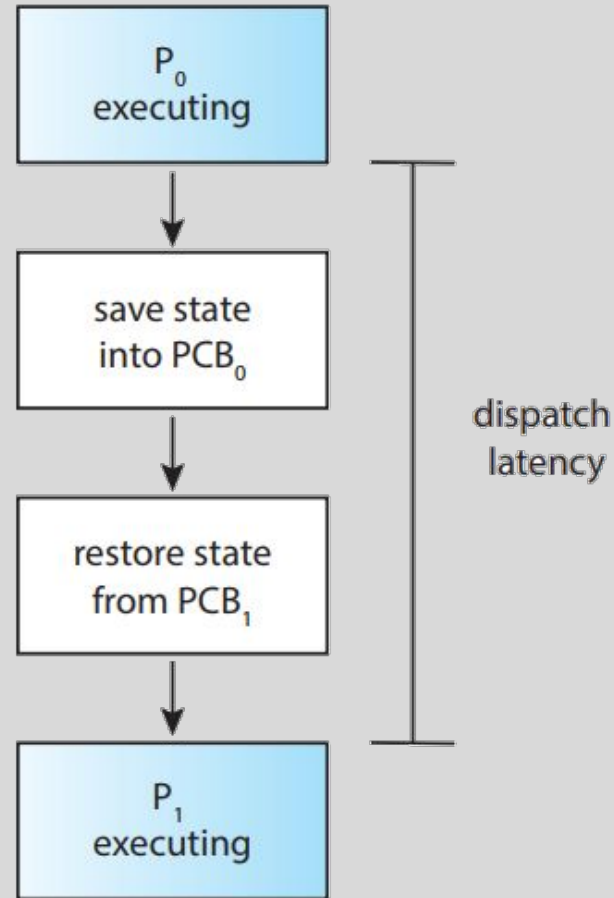
Latencia de despacho

El sistema no avanza en el trabajo útil del proceso saliente ni del entrante. Es el costo de reorganizar la ejecución.

Cuanto más frecuentes sean los cambios de contexto, más importante se vuelve ese costo.

Un *trade-off* muy importante de la planificación:

- Cambiar seguido de proceso puede mejorar la capacidad de respuesta
- Pero también puede aumentar la sobrecarga administrativa



The role of the dispatcher



Cambios de contexto voluntarios e involuntarios

Un cambio de contexto voluntario ocurre cuando el propio proceso deja de poder continuar usando la CPU y la libera voluntariamente. Por ejemplo:

- Solicita una operación de E/S
- Espera un recurso
- O se bloquea por alguna razón (ej. *sleep*)

En ese caso el sistema aprovecha para asignar la CPU a otro.



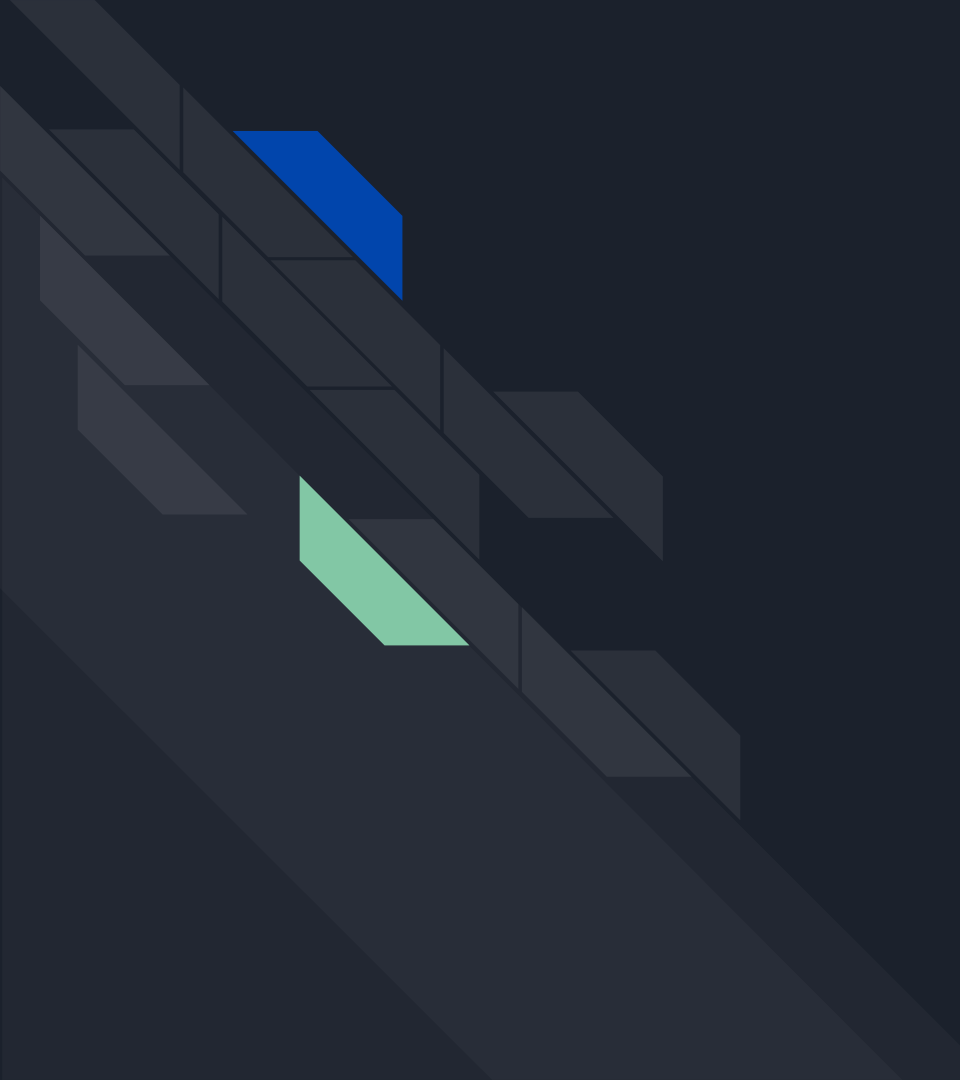
Cambios de contexto voluntarios e involuntarios

Un cambio de contexto involuntario ocurre cuando el SO le quita la CPU a un proceso que todavía podría seguir ejecutando. Por ejemplo:

- Expiró su quantum
- Llegó una entidad de mayor prioridad
- O porque la política de planificación así lo decidió

Notar que no todos los cambios de contexto reflejan el mismo tipo de decisión:

- Algunos ocurren porque el proceso ya no puede seguir
- Otros porque el sistema prefiere dar lugar a otro proceso



CRITERIOS DE PLANIFICACIÓN



Cómo evaluar un algoritmo de planificación

¿Cómo sabemos si un algoritmo es bueno o malo?

La respuesta no es única, porque distintos sistemas pueden priorizar objetivos diferentes.

Por ejemplo:

- En un sistema interactivo suele importar mucho que la respuesta al usuario sea rápida
- En un sistema por lotes puede importar más completar mucho trabajo total
- En otros casos puede interesar que ningún proceso espere demasiado

Los algoritmos de planificación no se comparan solo por su definición, sino también por los resultados que producen bajo ciertos criterios de evaluación.



Criterios de planificación

- La **utilización de CPU**, busca mantener el procesador ocupado la mayor cantidad de tiempo posible.
- El **throughput** o rendimiento, mide cuántos procesos se completan por unidad de tiempo.
- El tiempo de retorno o **turnaround time**, representa el tiempo total transcurrido desde que un proceso entra al sistema hasta que termina. Incluye espera, ejecución y también períodos de E/S.
- Uno de los más importantes, el **tiempo de espera**, se refiere al tiempo acumulado que el proceso pasa en la cola de listos esperando CPU.
- Y el **tiempo de respuesta**, que mide cuánto tarda un proceso en empezar a responder, no necesariamente en completarse.



Criterios de planificación

En general, se busca:

- Maximizar utilización y throughput,
- Minimizar tiempos de retorno, espera y respuesta.

Sin embargo, esos objetivos no siempre pueden optimizarse todos al mismo tiempo.



Trade-offs entre criterios y algoritmos

Dependiendo el algoritmo, cada uno privilegia cosas distintas:

- Simplicidad
- Menor espera promedio
- Mejor respuesta
- Mayor equidad
- O prioridad según contexto

Por eso, la elección del algoritmo depende del contexto:

- Tipo de sistema
- Tipo de carga
- Importancia de la interacción
- Necesidad de equidad
- Costo aceptable de sobrecarga

ALGORITMOS DE PLANIFICACIÓN





Planificación de la CPU y Algoritmos de Selección

Aunque las arquitecturas actuales suelen ser multinúcleo, para facilitar la comprensión de los algoritmos base, asumiremos un entorno de un solo núcleo donde solo se puede ejecutar un proceso a la vez.

Para evaluar y comparar la eficiencia de los algoritmos que veremos a continuación, utilizaremos los siguientes criterios:

- **Alcance:** Se analizará únicamente una ráfaga de CPU (CPU burst) por cada proceso, medida en milisegundos.
- **Métrica de comparación:** El rendimiento se determinará mediante el **tiempo de espera promedio** (Average Waiting Time), buscando siempre minimizar el tiempo que los procesos permanecen en la cola antes de ser atendidos.



FCFS: idea básica

First-Come, First-Served es la política de planificación más simple. El primer proceso que llega a la cola de listos es el primero en recibir la CPU. Conceptualmente puede pensarse como una cola FIFO:

- El primer proceso en entrar es el primero en salir.

Cuando un proceso obtiene la CPU, la conserva hasta que termina su ráfaga de CPU o hasta que se bloquea. Claramente, FCFS es un algoritmo **no apropiativo**.

La principal ventaja es su simplicidad:

- Fácil de entender
- Fácil de implementar
- Y aparentemente “justo” en el sentido de respetar el orden de llegada.

Pero esa simplicidad no implica buen rendimiento.

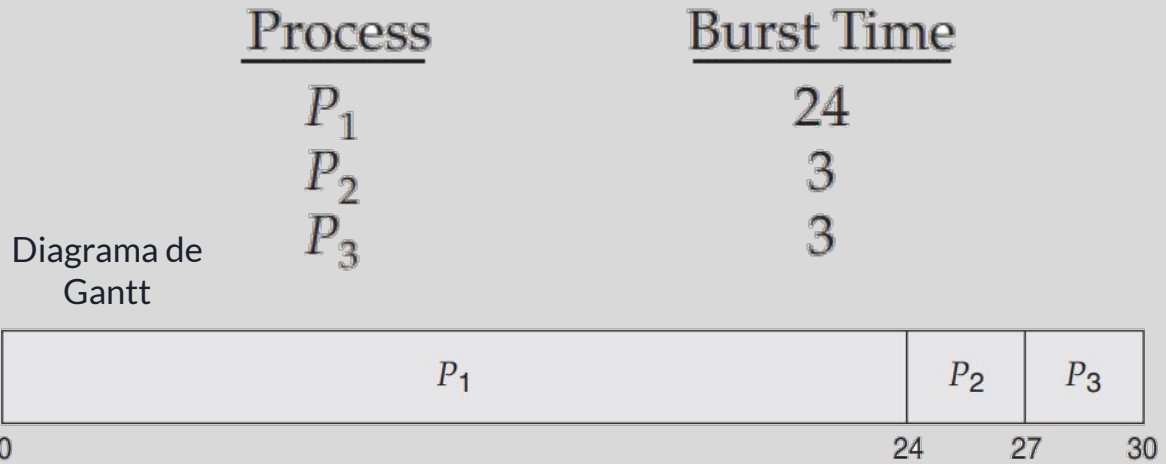
FCFS: ejemplo y limitaciones

Si un proceso largo llega primero:

- Ocupa la CPU durante mucho tiempo
- Y los procesos cortos deben esperar

Consecuencia:

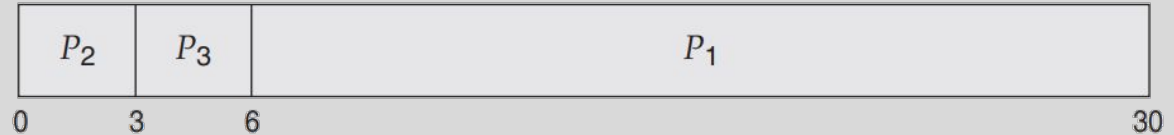
- Puede aumentar mucho el tiempo de espera promedio
- Y ofrecer mala respuesta en sistemas interactivos



El tiempo de espera para P_1 es de 0 ms, para P_2 , 24 ms y para P_3 , 27 ms.

Así, el **tiempo de espera promedio** es $(0 + 24 + 27)/3 = 17$ ms.

En cambio, si los procesos llegan en orden P_2 , P_3 , P_1 :



El **tiempo de espera promedio** ahora es $(6 + 0 + 3)/3 = 3$ ms.



SJF: idea básica

Shortest-Job-First propone una idea distinta: en lugar de respetar el orden de llegada, selecciona el proceso cuya próxima ráfaga de CPU es la más corta.

La intuición detrás de este algoritmo:

- Si primero se ejecutan trabajos cortos, se libera rápido la CPU y se evita que muchos procesos pequeños queden esperando detrás de uno largo.

SJF intenta mejorar el tiempo de espera promedio favoreciendo tareas que pueden completarse en menos tiempo.

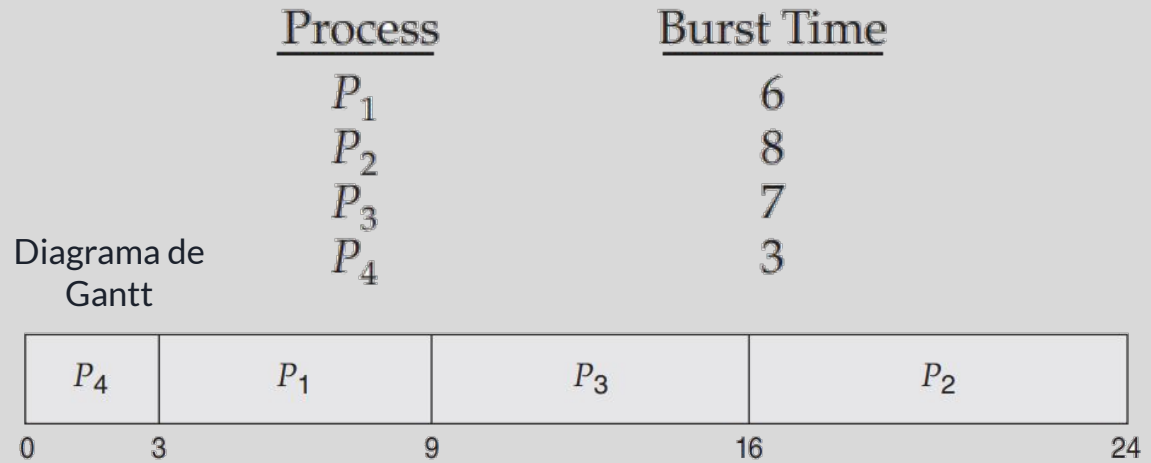
Cuando dos procesos tienen la misma duración estimada (empate), se usa FCFS.

Conceptualmente, SJF es un mejor algoritmo porque introduce una política más “inteligente”.

SJF: ejemplo

Los procesos más cortos se ejecutan primero y, como terminan rápido, dejan libre la CPU sin que otros procesos pequeños deban esperar detrás de trabajos largos.

Eso suele reducir el tiempo de espera promedio del conjunto, dado que dar lugar primero a trabajos cortos suele beneficiar al promedio del sistema.



Para P_1 , el tiempo de espera es de 3 ms, para P_2 , de 16 ms, para P_3 , 9 ms y para el proceso P_4 , 0 ms. El tiempo de espera promedio es:
 $(3 + 16 + 9 + 0)/4 = 7$ **milisegundos**.

En comparación, si usáramos el esquema de planificación **FCFS**, el tiempo de espera promedio sería de **10,25 ms**.



SJF: por qué minimiza la espera promedio

Idea intuitiva: si un proceso corto queda detrás de uno largo, su espera aumenta mucho en proporción a lo poco que necesitaba ejecutar.

En cambio, si adelantamos el corto y retrasamos al largo, el perjuicio para el largo suele ser menor que el beneficio obtenido por el corto.

Esto hace que SJF sea muy atractivo desde el punto de vista teórico:

- Es una política óptima respecto de una métrica específica.

Ahora bien, esa optimalidad tiene una limitación muy fuerte:

- Es necesario conocer la duración de la próxima ráfaga de CPU de cada proceso.

Acá aparece su gran problema práctico.



El problema práctico de SJF

El SO no puede conocer con certeza la duración futura de la próxima ráfaga de CPU de un proceso. Por definición no está disponible de manera exacta al momento de decidir.

La solución práctica consiste en **estimar** la duración de las próximas ráfagas a partir del comportamiento pasado del proceso.

Es decir, el sistema no sabe exactamente cuánto durará la próxima ráfaga, pero puede intentar **predecirla** usando información histórica. Y su calidad depende de qué tan buenas sean esas estimaciones.



SJF apropiativo: SRTF

Vimos SJF en una versión no apropiativa, donde el proceso elegido conserva la CPU hasta terminar su ráfaga actual.

Pero también existe una versión apropiativa, *Shortest-Remaining-Time-First* o SRTF.

En SRTF, el sistema no solo considera la duración esperada de la próxima ráfaga, sino el tiempo restante del proceso que está ejecutando. Si llega un nuevo proceso cuya ráfaga estimada es menor que el tiempo restante del actual, entonces el sistema puede apropiarle la CPU al que estaba ejecutando y darle lugar al nuevo.

La ventaja potencial es que puede reducir aún más ciertos tiempos de espera y respuesta.

A costa de introducir más interrupciones, más cambios de contexto y una dependencia aún mayor de buenas estimaciones.

SRTF: ejemplo

Si durante la ejecución de un proceso llega otro con menor tiempo restante:

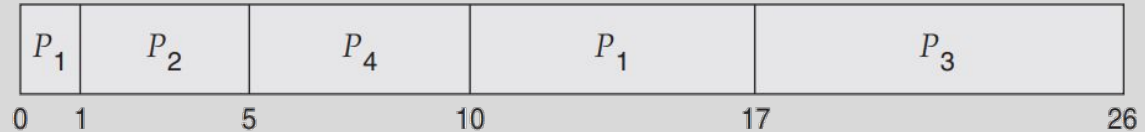
- El actual puede ser interrumpido
- El nuevo pasa a ejecutar

Resultado:

- Menor espera promedio en muchos casos
- Más cambios de contexto

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

Diagrama de Gantt



P_1 inicia en t_0 . P_2 llega en t_1 . En ese instante, el tiempo restante de P_1 (7 ms) es mayor que el tiempo requerido por P_2 (4 ms), entonces se interrumpe el proceso P_1 y se planifica el proceso P_2 . El tiempo de espera promedio para este ejemplo es $[(10 - 1) + (1 - 1) + (17 - 2) + (5 - 3)]/4 = 26/4 = 6,5$ ms.

La planificación SJF no apropiativa resultaría en un tiempo de espera promedio de 7,75 ms.



Round Robin: idea básica

Busca repartir la CPU de una manera más equilibrada entre los procesos listos.

En lugar de dejar que un proceso conserve la CPU hasta terminar su ráfaga, el sistema le asigna un pequeño intervalo de tiempo llamado *quantum*.

Mientras un proceso ejecuta, corre un temporizador.

- Si termina o se bloquea antes de agotar su *quantum*, libera la CPU voluntariamente.
- Si sigue ejecutando cuando el quantum expira, el sistema lo interrumpe, lo retira de la CPU y lo coloca al final de la cola de listos.

Así, la CPU va rotando entre los procesos listos, asignándole a cada uno turnos breves.

Claramente *Round Robin* un algoritmo **apropiativo**.

Su objetivo es ofrecer mejor capacidad de respuesta para sistemas interactivos, porque evita que un solo proceso retenga la CPU durante demasiado tiempo.

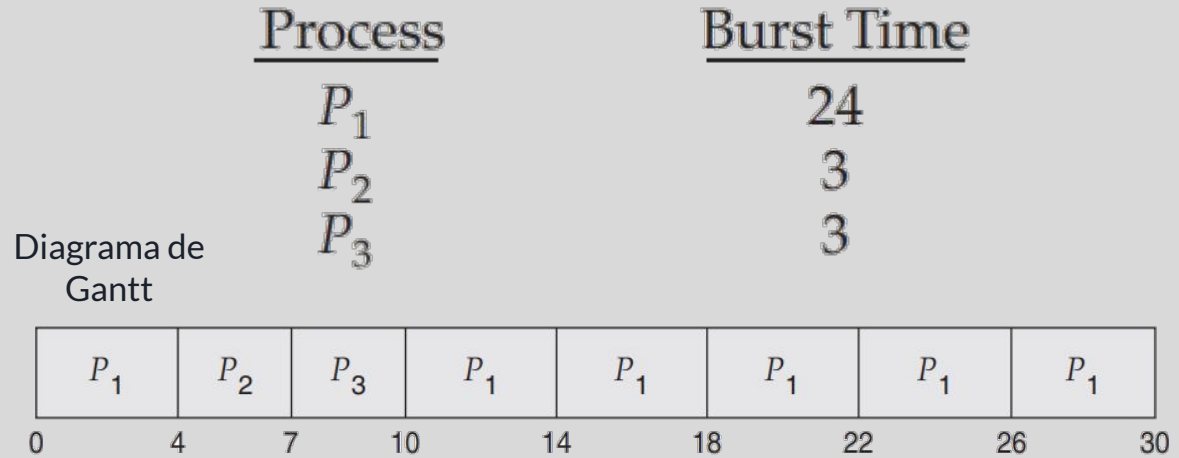
RR: ejemplo

Con un quantum fijo de 4ms:

- Cada proceso ejecuta por turnos
- Si no termina, vuelve al final de la cola
- Y la CPU rota entre los procesos listos

Resultado:

- Mejor respuesta para tareas interactivas
- Más cambios de contexto



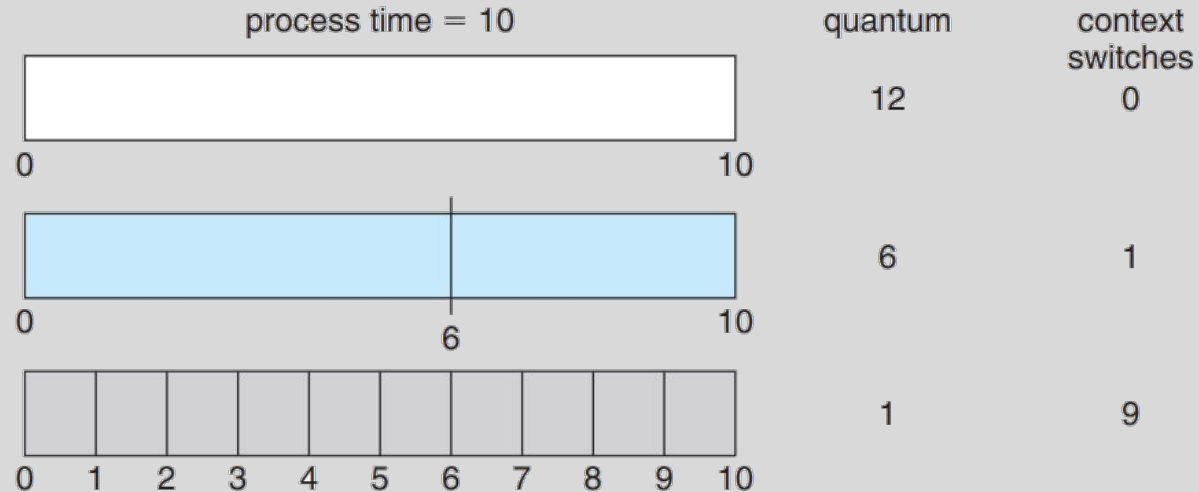
El tiempo de espera promedio para esta planificación es: P1 espera 6 ms (10 - 4), P2 espera 4 ms y P3 espera 7 ms. Así, el tiempo de espera promedio es $17/3 = 5,66$ ms.

RR y el valor del quantum

El comportamiento de RR depende del valor elegido para el quantum.

Si el quantum es muy grande, el algoritmo se parece cada vez más a FCFS.

Si el quantum es muy chico, la CPU rota muy seguido entre procesos. Mejora la capacidad de respuesta, pero produce gran cantidad de cambios de contexto.

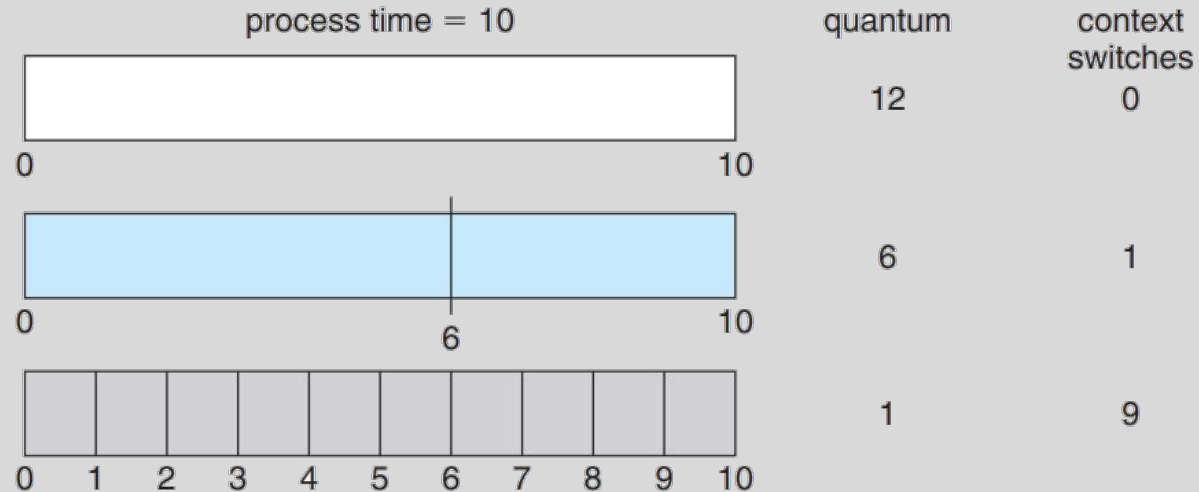


How a smaller time quantum increases context switches.

RR y el valor del quantum

Trade-off: Capacidad de respuesta vs Sobrecarga (*overhead*)

- Quantum grande reduce overhead, pero empeora respuesta
- Quantum chico mejora respuesta, pero aumenta sobrecarga



How a smaller time quantum increases context switches.



Planificación por prioridad: idea general

Acá no se selecciona al proceso por orden de llegada ni por duración estimada, sino por una prioridad asignada.

Regla general:

- La CPU se asigna al proceso con mayor prioridad.

Si varios procesos tienen la misma prioridad, puede aplicarse otro criterio como FCFS o RR.

SJF puede entenderse como un caso especial de planificación por prioridad:

- Si definimos que el proceso con ráfaga más corta tiene mayor prioridad, es una forma particular de prioridad.

La planificación por prioridad puede implementarse tanto de forma apropiativa como no apropiativa.

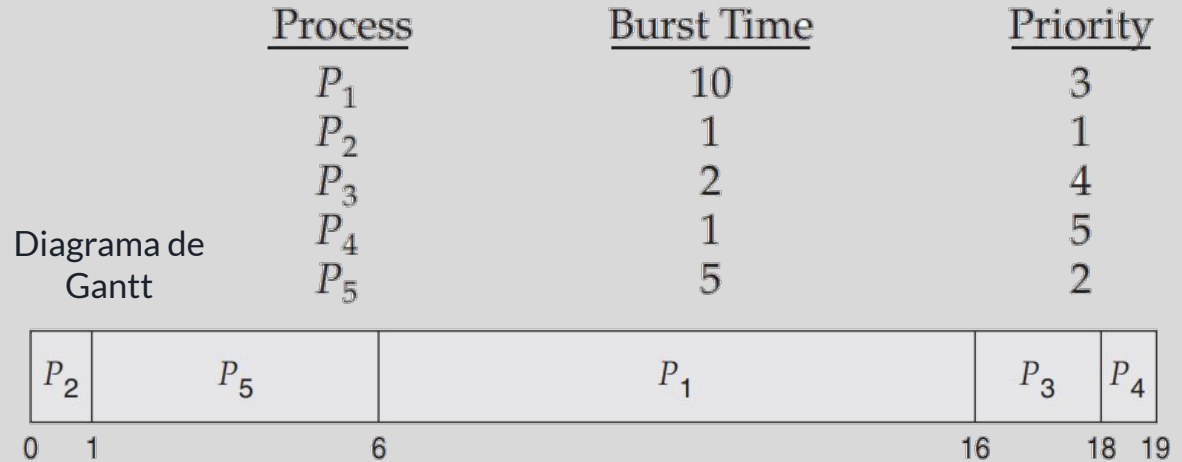
Prioridades internas y externas

La prioridad no aparece sola: alguien la define o la calcula.

A veces la prioridad se calcula a partir de características internas del propio sistema o del proceso.

Por ejemplo:

- Uso esperado de CPU
- Comportamiento de E/S
- Consumo de memoria
- Tipo de proceso
- O algún otro indicador medible



El tiempo de espera promedio es de **8,2 ms**.

Las prioridades pueden definirse interna o externamente.

Las prioridades internas utilizan una o varias cantidades medibles para calcular la prioridad de un proceso. Por ejemplo, los límites de tiempo, los requisitos de memoria, el número de archivos abiertos y la relación entre la ráfaga promedio de E/S y la ráfaga promedio de CPU.

Las prioridades externas se establecen mediante criterios externos al sistema operativo, como la importancia del proceso, el tipo y la cantidad de fondos que se destinan al uso de la computadora, el departamento que patrocina el trabajo y otros factores, a menudo políticos.

La planificación de prioridades puede ser apropiativa o no apropiativa.

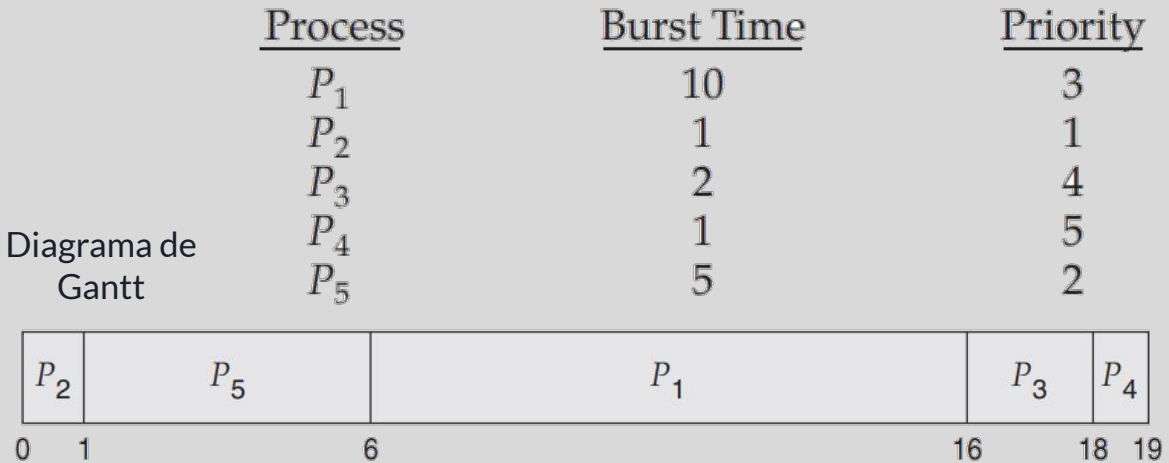
Prioridades internas y externas

En otros casos, la prioridad puede provenir de decisiones externas al SO.

Por ejemplo:

- Importancia funcional del proceso
- Clase de usuario
- Política de negocio
- Criticidad del servicio
- O decisiones administrativas

Diagrama de Gantt



El tiempo de espera promedio es de **8,2 ms**.

Las prioridades pueden definirse interna o externamente.

- Las prioridades internas utilizan una o varias cantidades medibles para calcular la prioridad de un proceso.
- Las prioridades externas se establecen mediante criterios externos al sistema operativo.

La planificación de prioridades puede ser apropiativa o no apropiativa.



Prioridad: riesgos, inanición y envejecimiento

La planificación por prioridad es muy flexible, pero introduce un riesgo importante: la inanición o *starvation*.

Esto es:

- Cuando procesos de baja prioridad quedan esperando indefinidamente porque siempre aparecen otros de prioridad superior que los desplazan.

Una forma clásica de mitigar este riesgo es el envejecimiento o *aging*.

La idea es aumentar gradualmente la prioridad de los procesos que llevan mucho tiempo esperando. Así, aunque inicialmente tengan baja prioridad, con el tiempo mejoran sus chances de obtener CPU.

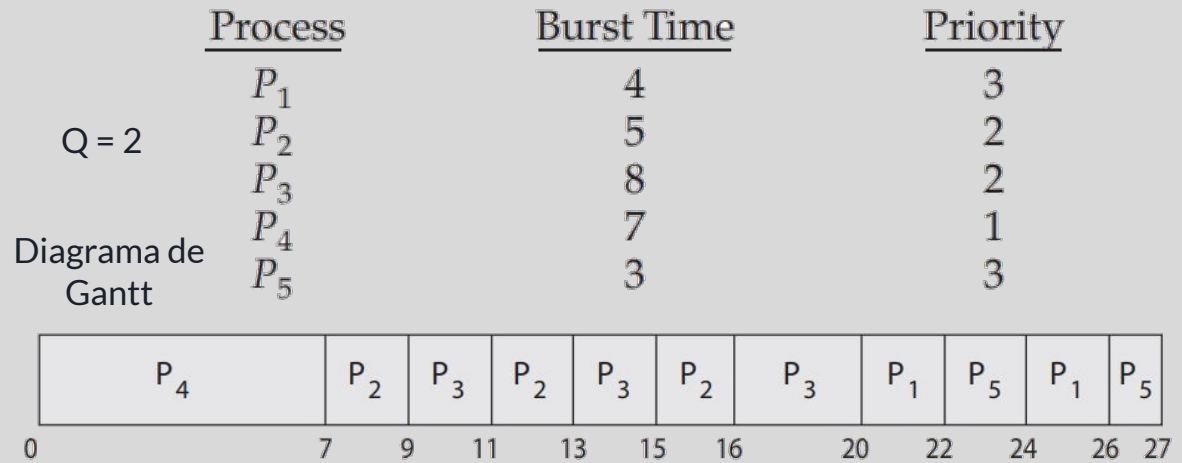
Prioridad + Round Robin

La lógica sería:

1. El sistema selecciona la cola o el conjunto de procesos de mayor prioridad
2. Entre los procesos de esa misma prioridad, reparte la CPU por turnos usando RR.

Esta combinación equilibra dos objetivos:

- Respeta la política de importancia entre niveles
- Evita que dentro de un mismo nivel, un proceso monopolice la CPU.



La prioridad organiza jerárquicamente el sistema, mientras que RR introduce reparto temporal entre pares.

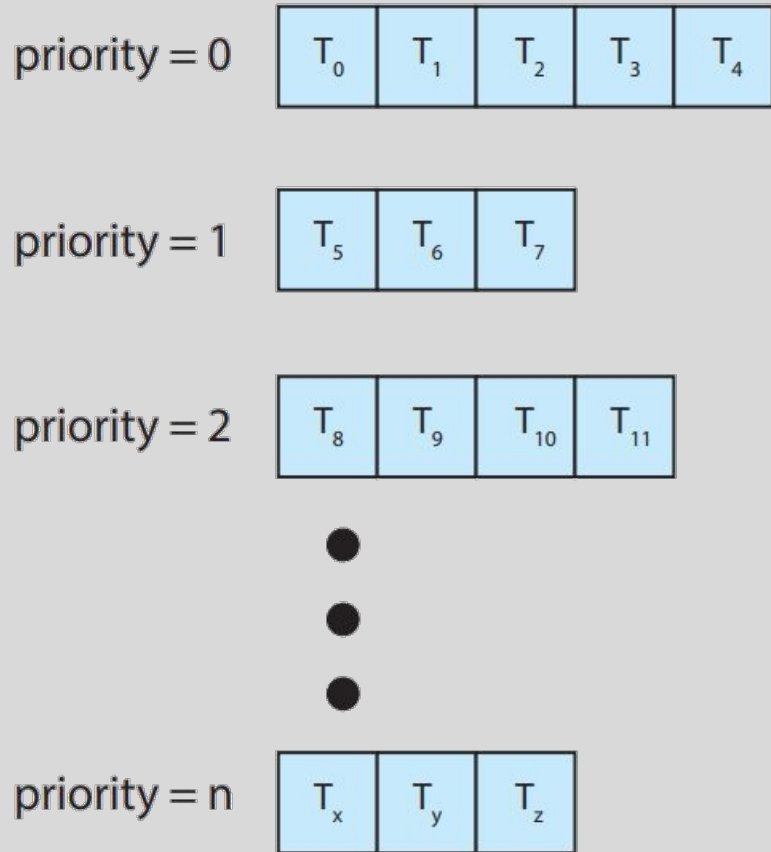
Colas multinivel

Las colas multinivel van un paso más allá en la idea de separar procesos según ciertas características.

En lugar de tener una única cola de listos para todos, el sistema organiza varias colas distintas.

Cada cola puede corresponder, por ejemplo, a:

- Diferente nivel de prioridad
- Distinto tipo de proceso
- Procesos interactivos
- Procesos por lotes
- Tareas de sistema
- Otros criterios



Separate queues for each priority.

Colas multinivel

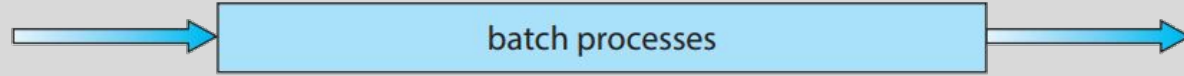
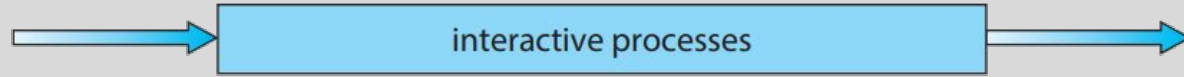
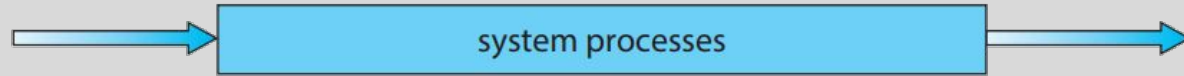
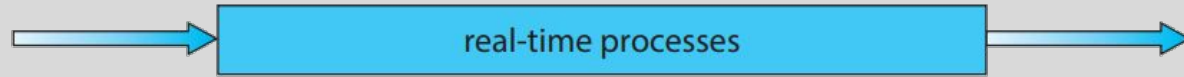
La selección ocurre en dos niveles:

1. Se decide entre las colas
2. Se aplica la política interna de la cola elegida.

Eso permite que distintas clases de procesos reciban tratamientos distintos. Por ejemplo, una cola puede usar RR y otra FCFS, según el tipo de carga que contenga.

La limitación de este esquema es que los procesos quedan asignados a una cola fija. Una vez clasificados, no cambian de grupo durante su ejecución.

highest priority



lowest priority

Multilevel queue scheduling.

Colas multinivel con retroalimentación

Las *feedback queues* corrigen la principal rigidez del esquema anterior.

Permiten que el proceso se mueva entre colas según su comportamiento.

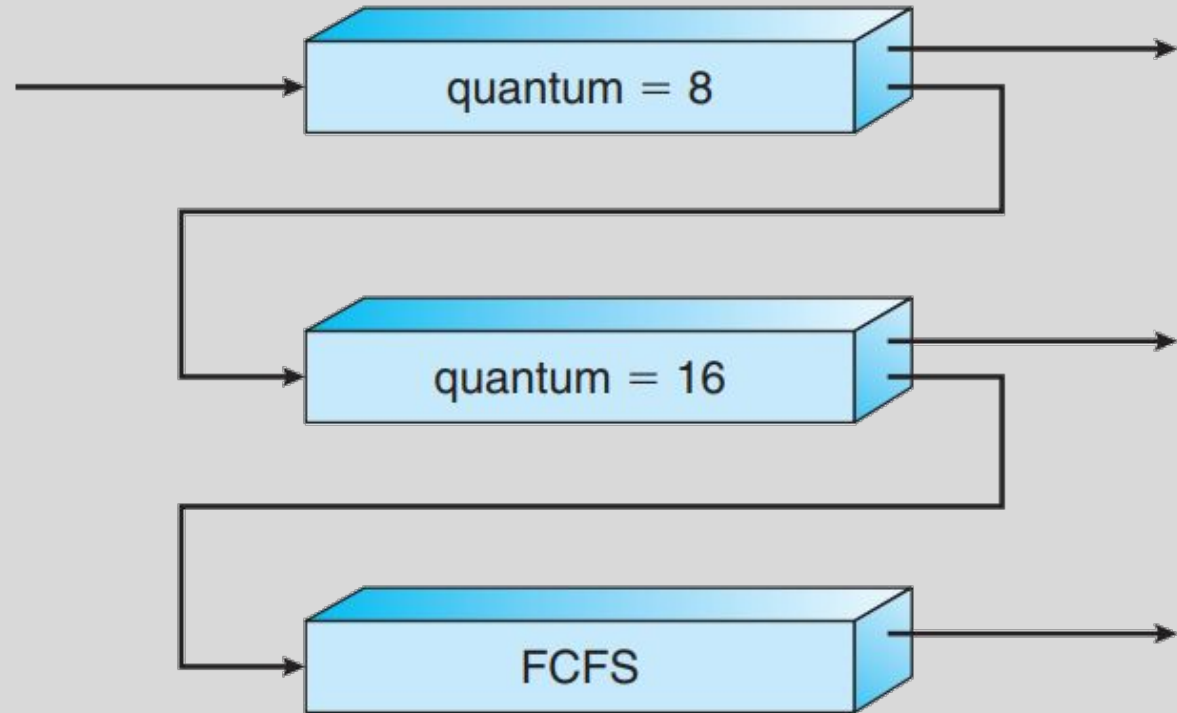
La idea general es adaptar la prioridad dinámicamente.

Por ejemplo:

Un proceso nuevo puede comenzar en una cola de mayor prioridad.

Si consume demasiada CPU, puede descender a una cola inferior.

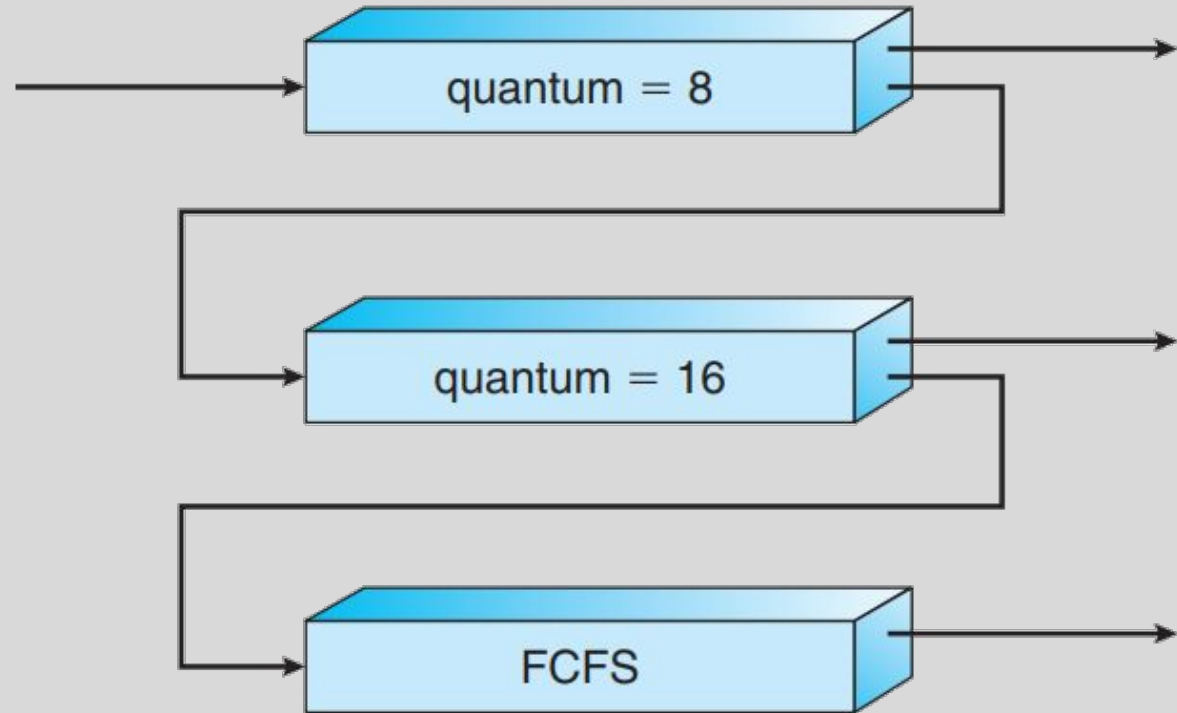
Si espera mucho tiempo o presenta otro patrón, podría volver a subir.



Multilevel feedback queues.

Colas multinivel con retroalimentación

Las colas con retroalimentación suelen considerarse entre las formas más potentes y flexibles de planificación clásica, aunque también son más complejas de diseñar y ajustar.



Multilevel feedback queues.



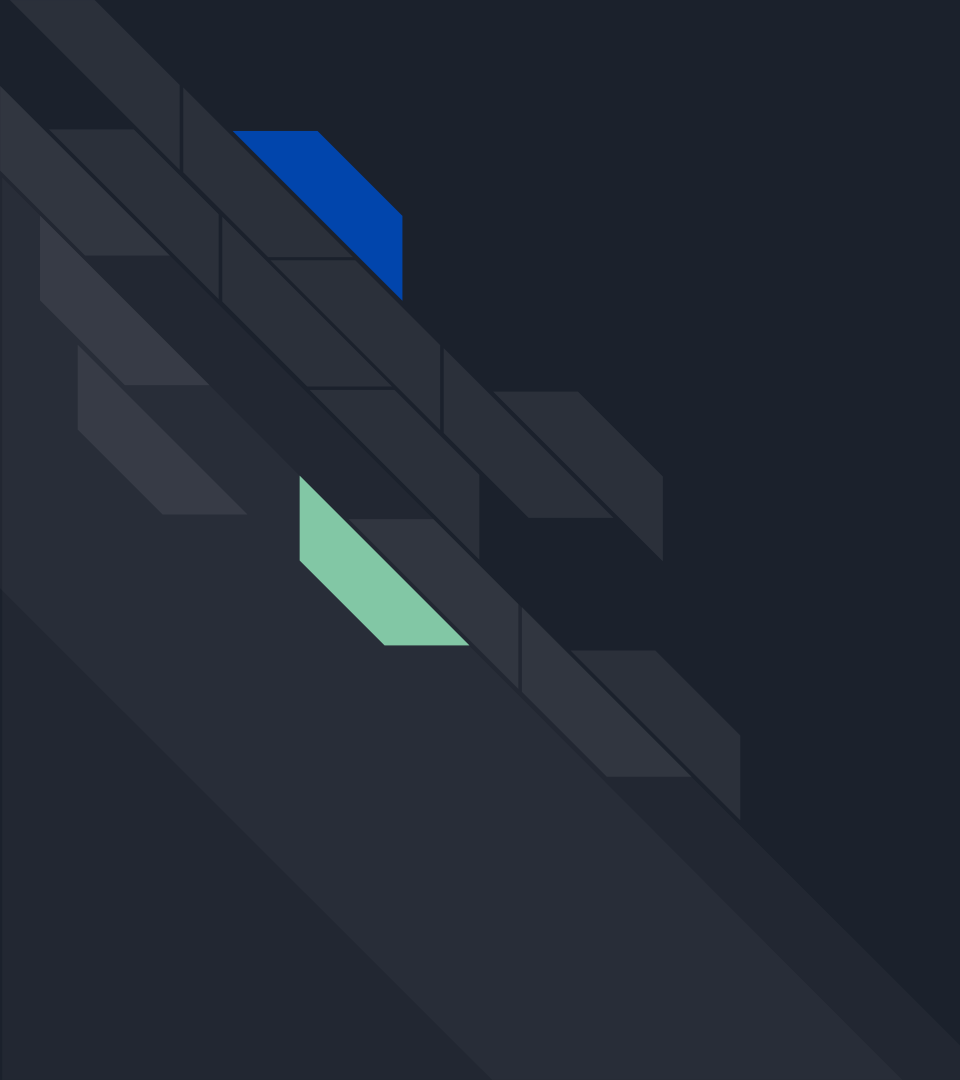
Resumen comparativo de algoritmos

Resumen comparativo

- FCFS → simple, pero puede esperar mucho
- SJF/SRTF → buena espera promedio, pero difícil de estimar
- RR → buena respuesta, más overhead
- Prioridad → flexible, riesgo de inanición
- MLQ / MLFQ → más adaptable, más complejo

Idea clave:

- Cada algoritmo favorece algo y sacrifica algo



PLANIFICACIÓN DE HILOS



Qué planifica realmente el sistema operativo moderno

Para introducir los algoritmos clásicos de planificación, hablamos en general de “procesos” como si fueran la unidad directa de planificación. Sin embargo, en los SO modernos la situación real suele ser un poco más precisa:

- lo que el kernel planifica directamente son, en general, **hilos de ejecución a nivel de kernel**.

La competencia por la CPU ya no se da solamente entre procesos distintos, sino también entre múltiples hilos que pertenecen incluso a una misma aplicación.

Los algoritmos de planificación siguen teniendo sentido. Lo que cambia es la unidad concreta sobre la que se aplican en la práctica.



Planificación de hilos

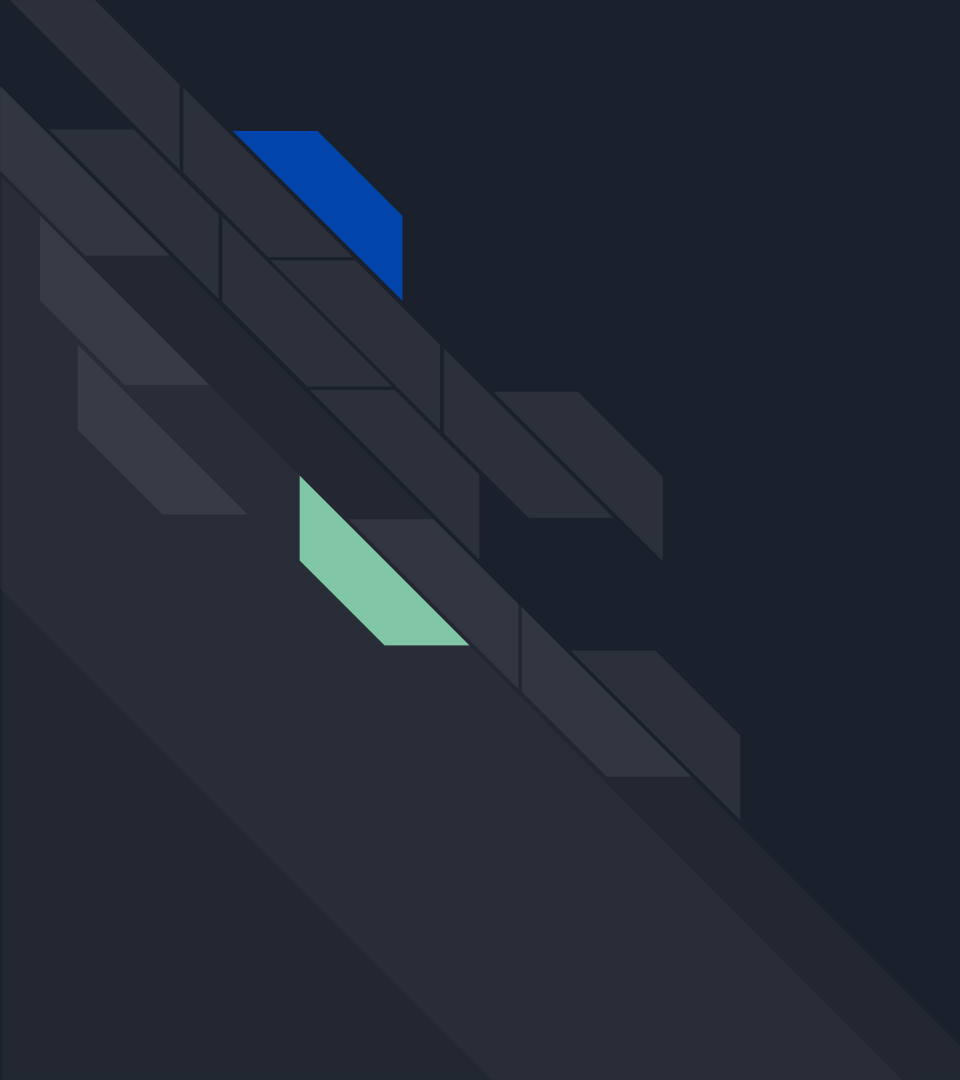
Cuando un sistema utiliza hilos, la planificación puede pensarse en dos niveles distintos.

- Por un lado, puede existir una librería de hilos en espacio de usuario que organiza qué hilo de usuario avanza dentro de una aplicación.
- Por otro lado, el kernel decide qué hilos visibles para él obtienen CPU real.

En sistemas actuales como Linux o Windows (modelos cercanos a uno a uno), el kernel planifica directamente hilos de kernel, y la competencia por la CPU ocurre a nivel global del sistema.

Esto significa que los algoritmos de planificación no desaparecen cuando aparecen hilos. Simplemente pasan a aplicarse sobre una unidad de ejecución más fina.

El problema de decidir quién ejecuta sigue existiendo, pero ahora muchas veces se formula en términos de hilos y no solo de procesos.



PLANIFICACIÓN MULTIPROCESADOR



Planificación en multiprocesadores

Hasta ahora, por simplificación, suponíamos una sola CPU disponible.

En sistemas con múltiples núcleos, la planificación se vuelve más compleja.

Ahora el SO ya no debe decidir solo quién ejecuta, sino también en qué núcleo se ejecuta. Es decir, el problema deja de ser solamente temporal y pasa a ser también espacial:

- No solo cuándo ejecutar, sino dónde ejecutar.

Esto agrega nuevas preguntas:

- ¿Todos los procesadores comparten una cola común?
- ¿Cada procesador tiene su propia cola?
- ¿Cómo se evita que algunos queden ociosos mientras otros están sobrecargados?
- ¿Cómo se mantiene un reparto razonable del trabajo?



Planificación en multiprocesadores

En enfoques antiguos o más simples, podía existir un procesador **maestro** que tomara decisiones de planificación y otros que solo ejecutaran trabajo. La desventaja de este enfoque es que el procesador maestro se convierte en un posible cuello de botella que puede reducir el rendimiento general del sistema.

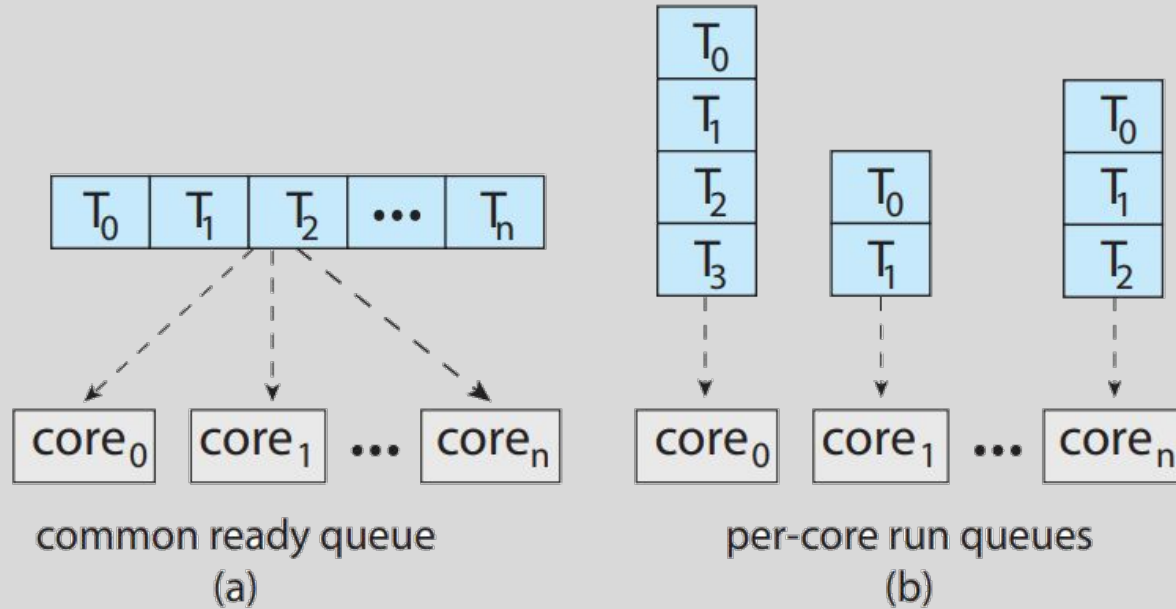
Pero el enfoque dominante hoy es el multiprocesamiento simétrico (SMP), donde varios procesadores participan activamente en la **planificación y ejecución**.

SMP: cola global vs colas por CPU

En sistemas *Symmetric Multiprocessing*, varios procesadores pueden participar activamente en la ejecución y planificación.

Existen dos estrategias:

1. Mantener una cola global compartida. Todos los procesadores toman trabajo de la misma cola de listos.
 - **Ventaja:** conceptualmente el trabajo está centralizado.
 - **Desventaja:** esa cola compartida puede convertirse en un punto crítico y requerir mucha coordinación.

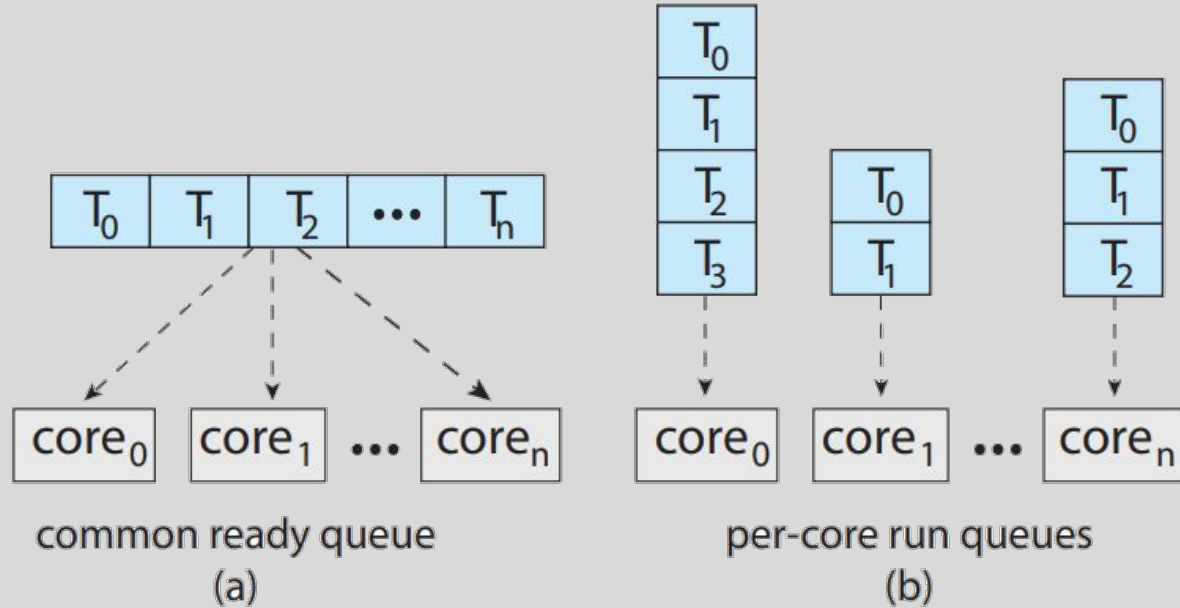


Organization of ready queues.

SMP: cola global vs colas por CPU

2. Tener colas separadas por CPU. Cada procesador administra su propia cola local. Esto mejora la escalabilidad, pero introduce otro problema:
 - El trabajo puede quedar mal distribuido entre procesadores si no se compensa de alguna manera.

Esta decisión tiene impacto directo en rendimiento y complejidad.



Organization of ready queues.

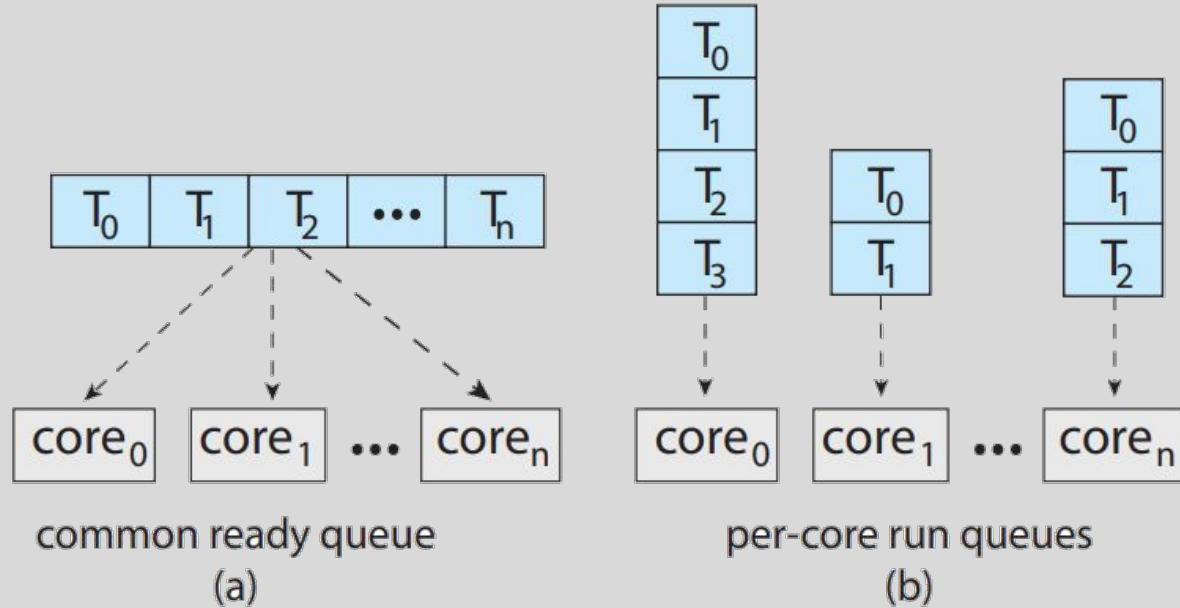
Balance de carga

Cuando cada núcleo tiene su propia cola, aparece un problema:

- Puede ocurrir que algunos procesadores tengan mucho trabajo pendiente, y otros poco o incluso queden ociosos.

Para corregir eso se utiliza **balance de carga**.

La idea general es redistribuir trabajo entre procesadores para que el uso del sistema quede más equilibrado.



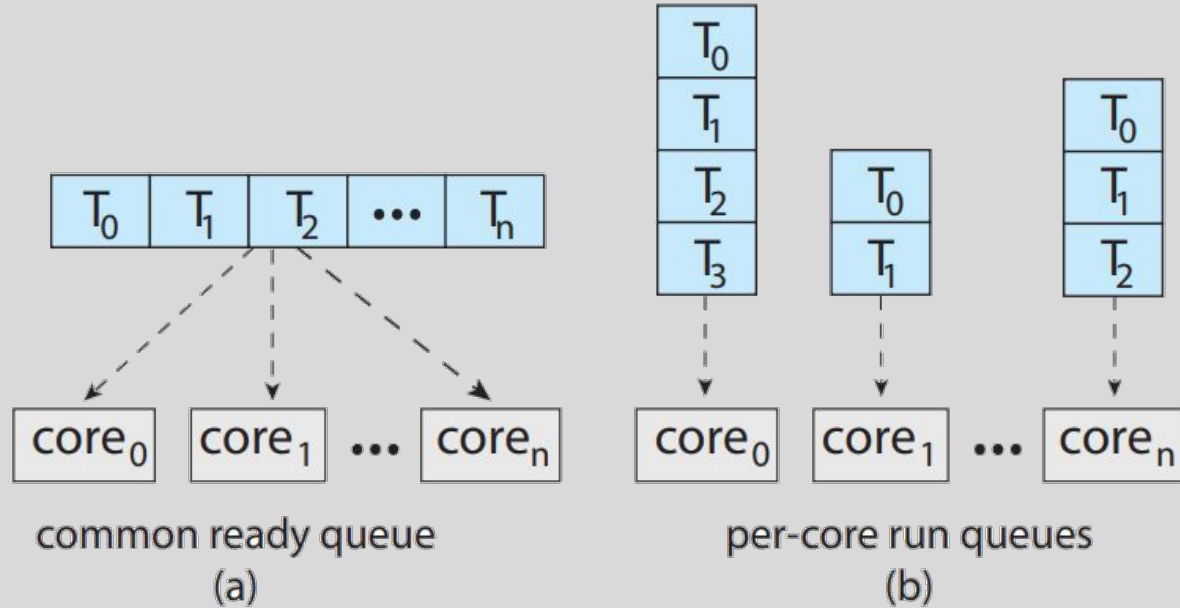
Organization of ready queues.

Balance de carga

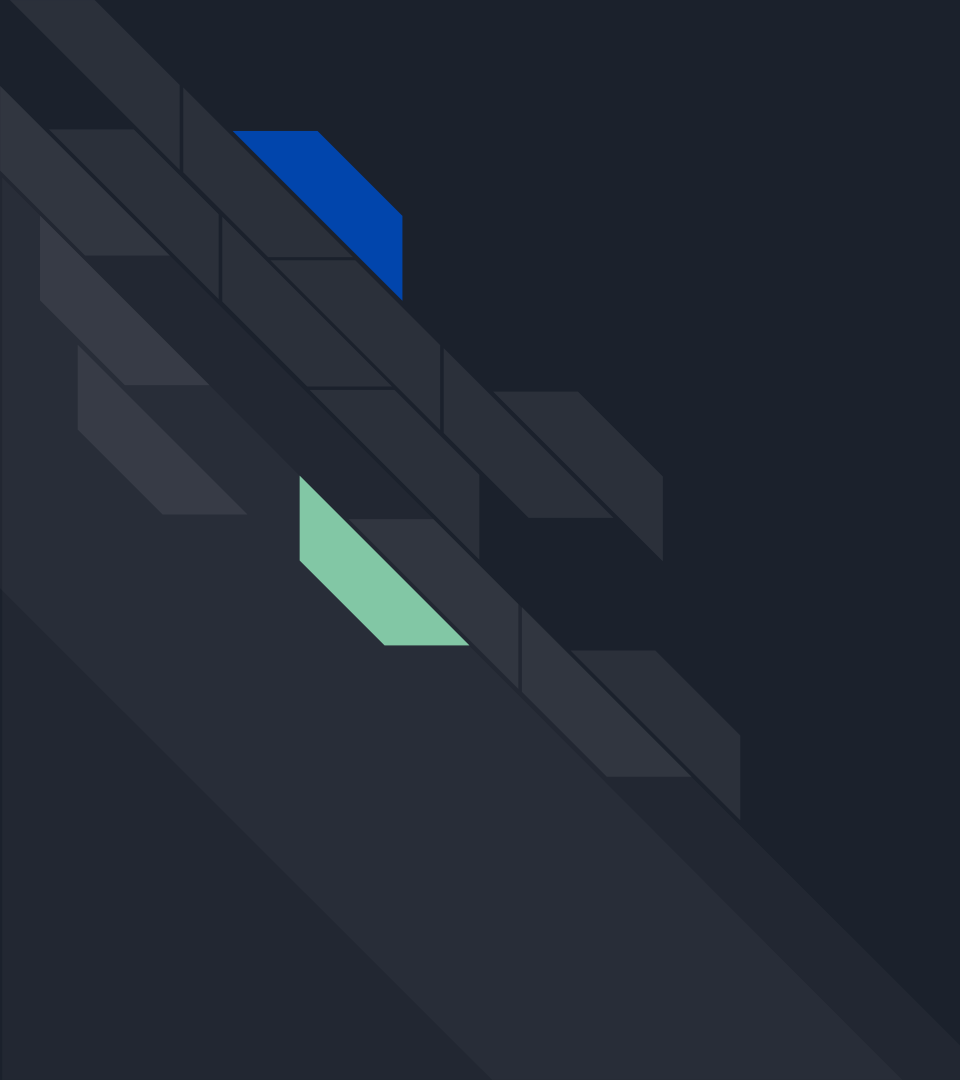
Esto puede hacerse de distintas maneras:

- Moviendo tareas desde procesadores más cargados hacia otros más livianos
- Permitiendo que un procesador busque trabajo cuando su cola se vacía
- Aplicando políticas periódicas de rebalanceo

En multiprocesadores, entonces, planificar no es solo seleccionar tareas: también es administrar cómo se distribuyen en el conjunto de CPUs disponibles.



Organization of ready queues.



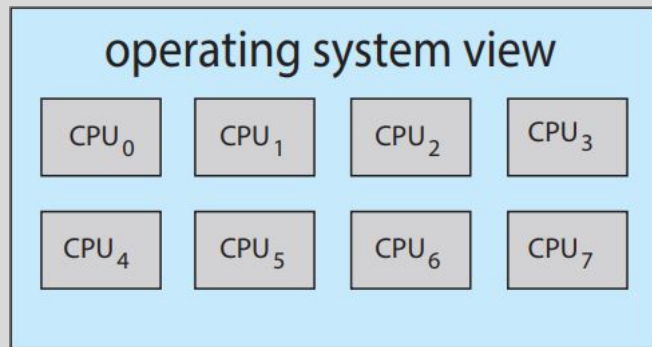
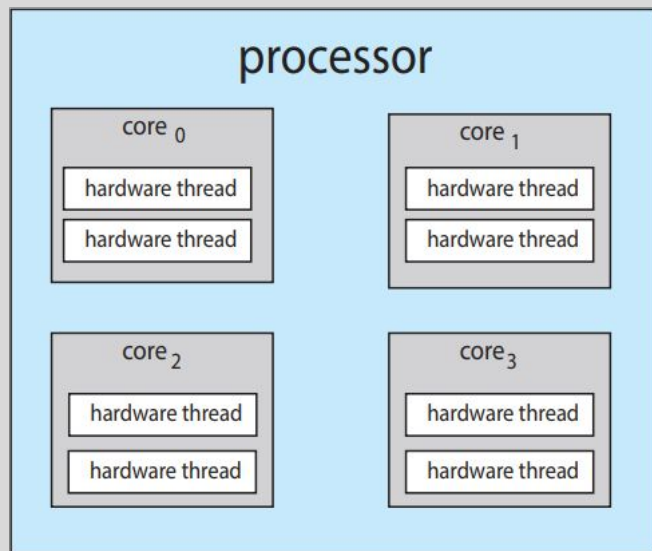
PROCESADORES MULTICORE

Multicore, CPUs lógicas e hyper-threading

En los sistemas actuales aparece otra capa de complejidad: la organización interna del propio hardware.

Un procesador moderno puede tener varios núcleos físicos, y cada núcleo puede exponer más de una CPU lógica al SO.

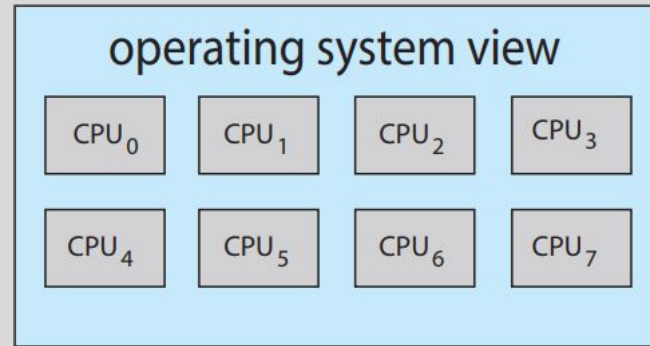
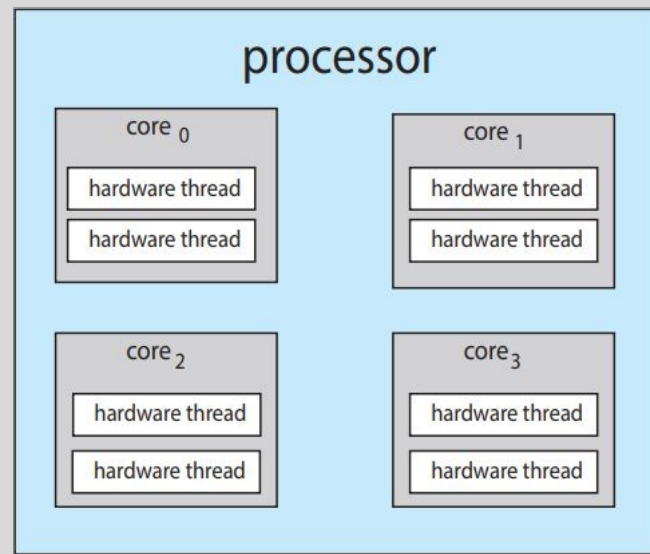
Desde la perspectiva del SO, cada CPU lógica aparece como una unidad disponible para ejecutar software.



Chip multithreading

Multicore, CPUs lógicas e hyper-threading

En procesadores Intel, por ejemplo, suele usarse el término *hyper-threading* para describir esta posibilidad de que un núcleo físico exponga múltiples hilos de hardware y, por lo tanto, múltiples CPUs lógicas al SO.

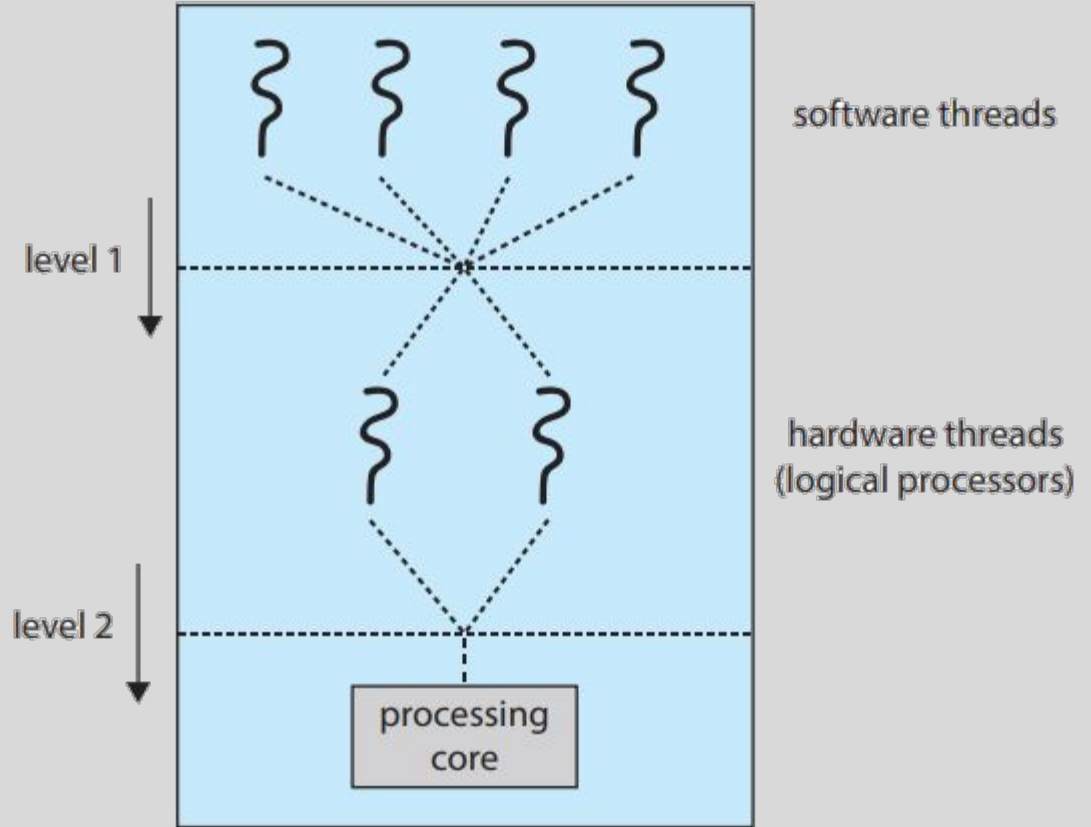


Chip multithreading

Planificación en Sistemas Multihilo y Multinúcleo

En los procesadores modernos, los recursos físicos de un núcleo (caché y pipelines) se comparten entre sus hilos de hardware. Dado que un núcleo solo puede ejecutar **un hilo de hardware a la vez**, se requieren dos niveles de decisión:

1. Primer Nivel (SO): El SO decide qué hilo de software se asignará a cada CPU lógica (hilo de hardware). En este nivel, el SO aplica algoritmos de planificación estándar (como RR o Prioridades).

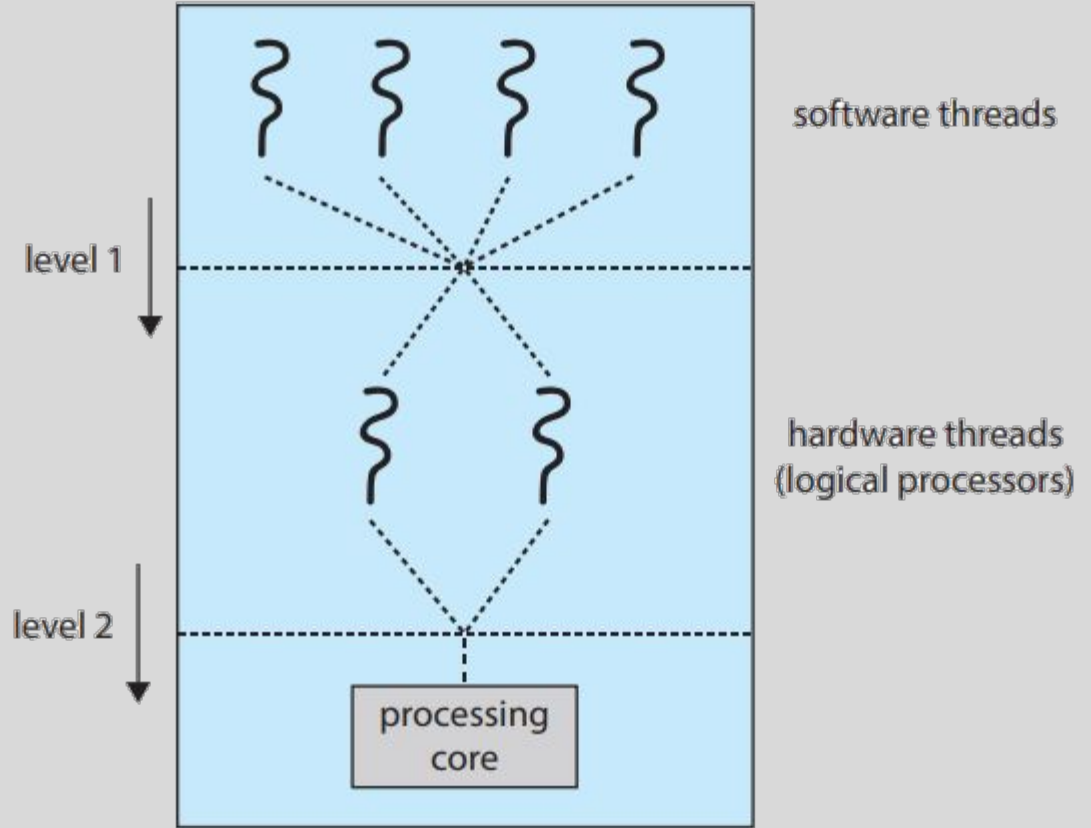


Two levels of scheduling

Planificación en Sistemas Multihilo y Multinúcleo

2. Segundo Nivel (Hardware/Core):
El núcleo físico decide cuál de sus hilos de hardware tiene acceso a los recursos de ejecución en un instante dado.

Para el segundo nivel, el hardware suele utilizar algoritmos simples y rápidos, como RR, para alternar el uso del núcleo entre los hilos de hardware disponibles de forma equitativa.



Two levels of scheduling

Muchas Gracias

Jeremías Fassi

Javier E. Kinter

